# Omni Pendulum Robot

A 2-DOF inverted pendulum balanced by a mecanum-wheeled AgileX LIMO in Gazebo Harmonic with ROS 2 Jazzy on Ubuntu 24.04

Project guide · step-by-step history, design decisions, theory and a line-by-line walkthrough of the code
Workspace: `~/omni_pendulum_ws` · Package: `omni_pendulum_sim`

# Contents


1. What was built — behaviour, architecture, topics, files
2. Step-by-step history — every stage, the problem found, what was changed and why
3. What happens behind the scenes — ROS 2, Gazebo, the bridge, time, physics
4. Control theory — the pendulum model, LQR, the cascade, the limits, measured results
5. Running, tuning and reference — commands, parameters, troubleshooting, supporting files
6. Line-by-line code walkthrough (6 files)
    1. balance_controller.py
    2. disturbance.py
    3. keyboard_nudge.py
    4. sim.launch.py
    5. robot.urdf.xacro
    6. setup.py

# Part 1 — What was built

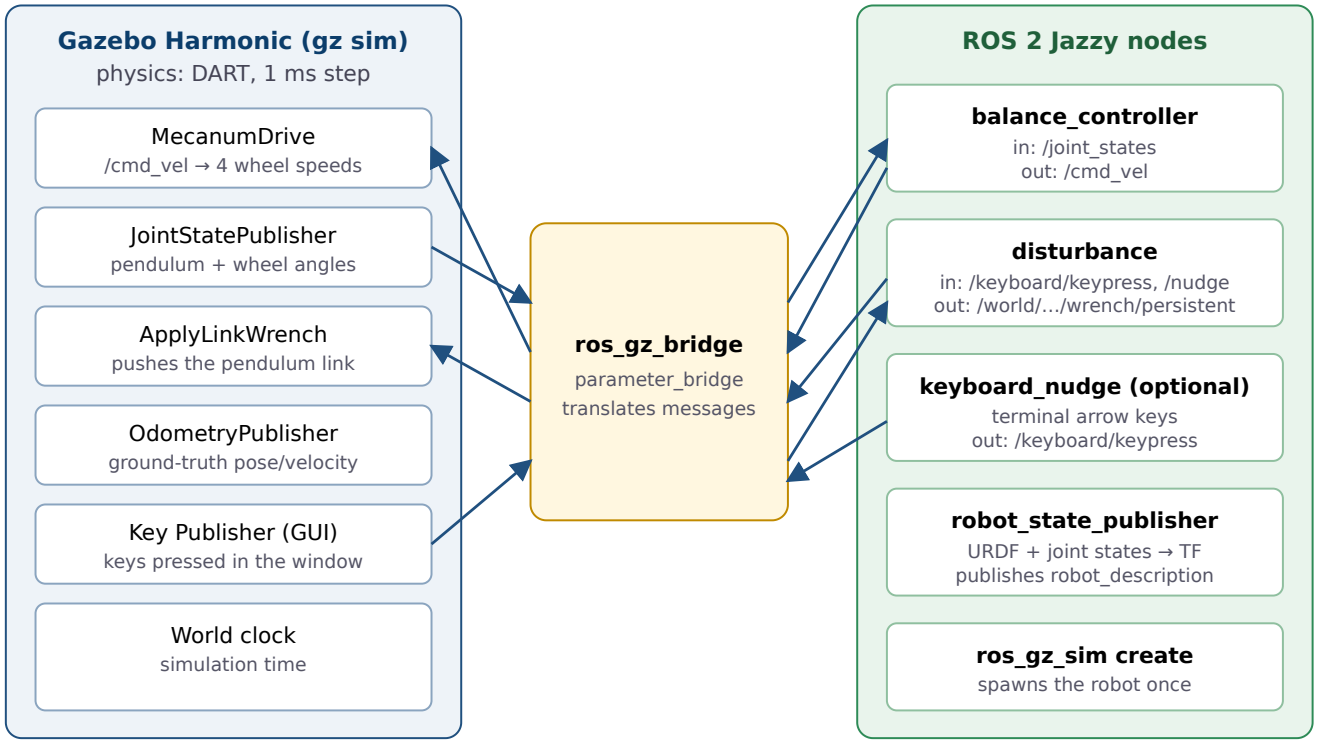
## 1.1 The finished behaviour

When you run `ros2 launch omni_pendulum_sim sim.launch.py`:

1. Gazebo opens with an AgileX LIMO robot (mecanum wheels) carrying a yellow platform. On the platform a 0.5 m red rod with a blue ball on top is mounted on a free two-axis hinge, so it can tip in any direction.
2. About 4 s after start a tiny 0.1 N nudge starts the pole falling. The robot does nothing while gravity tips it, so you can see it fall freely.
3. When the pole reaches 10° the controller engages: the robot drives in the direction of the fall to get back underneath the pole and stands it up again.
4. Every 4–8 s a random "human finger" nudge (0.3–1.0 N) hits the pole in a random direction and the robot recovers. You can add your own nudges with the arrow keys (in the Gazebo window or in a terminal running `keyboard_nudge`) and change their strength with `+` / `-`.
5. After a nudge the robot brakes gently (the pole never leans more than about 5° the other way), then drifts back to its start point at no more than 0.3 m/s. Its speed never exceeds 2 m/s.

## 1.2 Architecture

Two separate systems work together. **Gazebo Harmonic** simulates the physics (gravity, contacts, friction, the robot and the pole). **ROS 2 Jazzy** runs the robot software (the controller and the disturbance generator). They use different communication systems (Gazebo Transport and ROS 2 DDS), so **ros_gz_bridge** translates messages between them.



## 1.3 Topics

| Topic | Type | From → To | Purpose |
|---|---|---|---|
| `/joint_states` | sensor_msgs/JointState | Gazebo → controller, robot_state_publisher | Pendulum pitch/roll angle and rate (the controller's only sensor) |
| `/cmd_vel` | geometry_msgs/Twist | controller → Gazebo MecanumDrive | Base velocity command (x forward, y left) |
| `/world/balance_world/wrench/persistent` | ros_gz_interfaces/EntityWrench | disturbance → Gazebo ApplyLinkWrench | Nudge torque on the pendulum (sent as changes) |
| `/world/balance_world/wrench/clear` | ros_gz_interfaces/Entity | disturbance → Gazebo | Remove leftover wrenches at start-up |
| `/keyboard/keypress` | std_msgs/Int32 | Gazebo Key Publisher or keyboard_nudge → disturbance | Arrow / + / - key codes (Qt codes) |
| `/nudge` | geometry_msgs/Vector3 | you (ros2 topic pub) → disturbance | Manual push force in newtons |
| `/cmd_vel_ref` | geometry_msgs/Twist | optional teleop → controller | Desired travel velocity while balancing |
| `/odom` | nav_msgs/Odometry | Gazebo MecanumDrive → ROS | Wheel odometry (cannot see slip) |
| `/ground_truth/odom` | nav_msgs/Odometry | Gazebo OdometryPublisher → ROS | True pose/velocity, used to measure slip |
| `/clock` | rosgraph_msgs/Clock | Gazebo → all nodes | Simulation time |

## 1.4 Files in the package

| File | Role |
|---|---|
| `omni_pendulum_sim/balance_controller.py` | Balancing controller node (LQR cascade with limits) |
| `omni_pendulum_sim/disturbance.py` | Human-nudge simulator node (random, keyboard, manual) |
| `omni_pendulum_sim/keyboard_nudge.py` | Terminal arrow-key input node |
| `launch/sim.launch.py` | Starts the whole simulation |
| `urdf/robot.urdf.xacro` | Robot model and Gazebo plugins |
| `worlds/balance_world.sdf` | World: ground, light, physics, ApplyLinkWrench system |
| `config/gui.config` | Gazebo window layout + Key Publisher + closer camera |
| `meshes/limo/*_lite.dae` | Simplified LIMO body and wheel meshes (installed); originals kept for reference |
| `setup.py`, `setup.cfg`, `package.xml`, `resource/` | ROS 2 Python package build and metadata files |

# Part 2 — Step-by-step history

The project was built in ten stages. Each stage started from something you saw in the simulation (or a request), was diagnosed with measurements, and was fixed. This part records the reasoning, including the mistakes found along the way, because they explain why the code looks the way it does.

### Step 1 Starting point: reviewing the first draft

**Situation.** The package `omni_pendulum_sim` already existed in `~/omni_pendulum_ws` with a first draft of the controller, URDF, launch file and setup.py (the files were not empty, as first thought). The launch file was named `sim.launch.py`, which is the standard ROS 2 naming.

**Problems found in the draft:**

- The controller mapped tilt directly to velocity with a PD law ( `v = Kp·θ + Kd·θ'` ). Part 4.2 shows mathematically that this can never balance an inverted pendulum.
- It measured time with the wall clock while the data came from simulation time, so its derivative was wrong.
- The wheels were plain cylinders. Gazebo's MecanumDrive plugin computes correct wheel speeds, but with ordinary friction the robot cannot slide sideways, so strafing (needed for roll balance) was impossible.
- package.xml had no runtime dependencies, and Gazebo topics were relative (could get model prefixes).

**What was done.** Anisotropic “mecanum” friction was added to every wheel, topics were made absolute, dependencies were declared, all nodes were switched to simulation time, and the controller was rewritten: it now computes a desired *acceleration* from LQR state feedback and integrates it into a velocity command. Gains were computed with `scipy` from the pendulum's real mass and length.

**Result.** In a headless test the plain robot balanced while driving forward, sideways and diagonally (max tilt 0.11 rad).

### Step 2 Replacing the plain robot with the AgileX LIMO

**Request.** Use the AgileX LIMO from GitHub, and make the pole actually fall (it stayed upright because nothing disturbed it).

**Findings.** AgileX's `limo_ros2` repository only has diff-drive and Ackermann models, written for the old Gazebo Classic (plugins such as `libgazebo_ros_diff_drive.so` do not exist in Gazebo Harmonic). There is no mecanum model.

**What was done.**

- The LIMO meshes and dimensions (wheelbase 0.2 m, track 0.13 m, wheel radius 0.045 m, mass 2.16 kg) were copied into the package; the mecanum behaviour was rebuilt for Gazebo Harmonic.
- The STL showed the meshes are in millimetres, the DAE in metres; the DAE was used.
- When URDF is converted to SDF, links joined by fixed joints are merged (“lumped”) into the root link, so `base_link` disappears into `base_footprint`. The wheel friction direction was therefore expressed in `base_footprint`.

- A custom world (`balance_world.sdf`) was created to load Gazebo's ApplyLinkWrench system so a node can push the pendulum, and a disturbance node was added.
- The pendulum got a ball on top (rod 0.5 m/0.1 kg, ball 0.1 kg), and the controller gained a position term (4-state LQR) so the robot returns home instead of drifting.

**Verification.** A test pushed the pole at 0°, 90° and −135° and logged the direction the robot drove: 2°, 94° and −132°, confirming it drives toward the fall to get back under the pole.

### Step 3 “The robot is missing in the Gazebo tree”

**Diagnosis.** A Gazebo server from an earlier run was still running in the background with no GUI or launch attached. The launch file started the Gazebo *server* and *GUI* as two processes; the new GUI connected to the first server it found, which was the wrong one.

**Fix.** Gazebo now runs as a single process containing both server and GUI (`gz sim -r world`), so the window always shows its own simulation; `gui:=false` runs server only. Lesson recorded in the guide: stop the simulation with Ctrl+C, and use `pkill -f "gz sim"` to clear leftovers.

> During this step one of the test runs opened a second Gazebo window while yours was running, which caused “Moved backwards in time” warnings. Afterwards all tests used an isolated `GZ_PARTITION` and `ROS_DOMAIN_ID` so they can never interfere with your simulation.

### Step 4 “The pendulum is floating mid-air”

**Diagnosis.** The pole's pivot had been placed from the LIMO's collision box, not from the visual mesh. Measuring the mesh vertices showed the LIMO's top is not flat: the centre is a dip at 0.1455 m, a sloped cover peaks at 0.2535 m, so a pivot at 0.26 m floated 11 cm above the body.

**Fix.** The pivot was moved onto the measured surface and the collision box (which in AgileX's file stood 5 cm above the real body) was lowered so it could not hit the pole.

### Step 5 Platform, and the “Force Quit / Wait” dialog

**Request.** Build a platform on the LIMO; Gazebo was freezing (“not responding”).

**Diagnosis.** The LIMO body mesh was 1,619 separate parts, each with its own material, 755,790 triangles (52 MB), and each of the four wheels another 140,290 triangles (11.4 MB). Loading and drawing that froze the GUI.

**Fix 1 — mesh simplification.** No mesh library was installed, so a small numpy tool was written: it merges all parts by colour (only 4 distinct colours exist) and decimates by *vertex clustering* (snap every vertex to a 2 mm grid, merge vertices in the same cell, drop triangles that collapse). Body: 755,790 → 41,255 triangles in 4 groups (2.8 MB). Wheel: 140,290 → 14,499 (1.0 MB). Total installed meshes went from 63.6 MB to 3.8 MB.

**Fix 2 — platform.** A 26 × 17 cm yellow plate on four posts, with its top at 0.266 m (just above the cover's peak), with collision, and the pole's puck and pivot on top of it.

### Step 6 Making the pole fall freely; wind

**Request.** The pole looked stiff; it should fall freely and the robot should keep adjusting.

**Findings.** With the controller off, the pole fell over in under a second, so the joints were free. It looked stiff because it never leaned more than 2–3°. Testing also showed Gazebo *adds* persistent wrenches together instead of replacing them, so the disturbance node was rewritten to send only the *change* in torque.

**What was done.** An `engage_tilt_rad` phase (robot waits until the pole has tipped) and a continuously varying random wind were added, and the gains were softened.

**Problem found.** Catching at 8° failed: logs showed the sideways command pinned at the 1.5 m/s speed limit while the tilt grew. Engaging at 5° and a 2 m/s limit fixed it.

### Step 7 Human nudges instead of wind; discovering the traction limit

**Request.** In real life the disturbance is a tiny nudge by a human.

**What was done.** The disturbance became a finger tap: a 0.15 s half-sine force pulse at a random height (0.3–0.5 m) on the rod, applied as the equivalent torque (force × height) because ApplyLinkWrench acts at the link origin (the pivot). A `/nudge` topic allows manual pushes.

**Problem found.** A sweep of nudge strengths showed diagonal pushes failed much earlier than straight ones, and stiffer gains made it *worse*. A ground-truth odometry plugin was added to compare commanded and real velocity. Logging a failing diagonal nudge showed the real robot lagging far behind the command (2.0 m/s commanded, 1.08 m/s real) and still moving at 1.09 m/s when the controller thought it had almost stopped. The controller was asking for 8 $m/s^2$, more than the wheels can deliver diagonally, so its internal velocity estimate was wrong and it braked too late.

**Fix.** The acceleration and velocity limits are now applied to the 2D vector (so diagonal moves keep their direction) and the acceleration limit was set to what the wheels can deliver. 4, 5, 6 and 8 $m/s^2$ were compared; 6 was best (Part 4.6).

### Step 8 Proving gravity; arrow-key nudges; larger visible fall

**Request.** The pole still felt stiff; show gravity acting on the bob; nudge with the arrow keys.

**Measurement.** Free fall from 1° to 90° took 0.881 s in simulation against 0.920 s from pendulum theory with a fixed pivot (Part 4.7); the small difference is the robot recoiling. Gravity was correct, so the “stiffness” was perception: a 5° catch seen from Gazebo's default camera 6 m away.

**What was done.**

- The engage angle was raised to 10° (15° was tested and cannot be caught).
- A custom GUI layout moves the camera to 1.6 m behind the robot and adds Gazebo's Key Publisher plugin.
- `keyboard_nudge.py` was written for terminal input. Testing through a pseudo-terminal revealed that Python's buffered stdin swallowed arrow-key escape sequences; it now reads raw bytes.
- Both inputs publish Qt key codes on `/keyboard/keypress`, so the disturbance node has a single code path.

### Step 9 Overshoot after strong nudges; speed limit

**Request.** After a strong nudge the robot moved so fast that the pole leaned the opposite way and fell. The speed should be just enough, with an upper limit.

**Diagnosis.** Measured: the opposite lean exceeded the original lean (e.g. 12° opposite after a 10° push). The cause is the “brake and return home” part of the controller, which works by tipping the pole backwards and had no limit; the hard 2 m/s speed clip also dropped the pole.

**Fix.** The LQR law was rewritten as an equivalent cascade (Part 4.5) where braking is a requested lean angle capped at 3.4°, return speed is capped at 0.3 m/s and 2 m/s remains an absolute limit. The opposite lean is now below 5° for every survivable nudge (Part 4.6).

### Step 10 Day-to-day issues

- **“Package not found”** — the terminal had sourced ROS but not the workspace; every terminal needs both `source /opt/ros/jazzy/setup.bash` and `source ~/omni_pendulum_ws/install/setup.bash`.
- **Yellow QML “Binding loop” warnings** at start-up come from Gazebo's own GUI files and are harmless.
- **When to rebuild.** With `colcon build --symlink-install`, edits to existing Python, launch, URDF, world and config files take effect on the next launch; rebuild only after adding new files or programs.

# Part 3 — What happens behind the scenes

## 3.1 Start-up sequence

1. `ros2 launch` runs `generate_launch_description()` in sim.launch.py.
2. `xacro` expands robot.urdf.xacro into plain URDF text (macros, properties and maths evaluated).
3. robot_state_publisher starts and publishes that text on `robot_description`.
4. Gazebo starts with balance_world.sdf and the custom GUI layout. Its world loads the Physics, UserCommands, SceneBroadcaster and ApplyLinkWrench systems.
5. The bridge starts and creates the ROS↔Gazebo topic pairs.
6. The controller and disturbance nodes start and wait for `/clock` and `/joint_states`.
7. After 2 s, `ros_gz_sim create` reads `robot_description`, Gazebo converts the URDF to SDF (lumping fixed joints, attaching the `<gazebo>` plugin blocks) and spawns the model. Its plugins (MecanumDrive, JointStatePublisher, OdometryPublisher) start publishing.
8. At about 4 s of simulation time the disturbance node sends the release nudge.

## 3.2 One control cycle

1. DART advances physics by 1 ms: gravity, contacts, joint motion.
2. JointStatePublisher sends the joint angles and rates; the bridge converts `gz.msgs.Model` to `sensor_msgs/JointState`.
3. `joint_callback` runs; if 5 ms of simulation time have passed it computes the control law and publishes a `Twist`.
4. The bridge converts it to `gz.msgs.Twist`; MecanumDrive applies mecanum inverse kinematics (each wheel speed = (vx ± vy ± (L+W)ω)/r) and commands the four wheel joints.
5. The wheels' anisotropic friction turns the wheel spin into body motion in any direction.

## 3.3 Simulation time

Gazebo may run slower or faster than real time. All nodes use `use_sim_time: True`, so their clock is `/clock` from Gazebo, and the controller computes `dt` from message timestamps. This keeps the control maths consistent with the physics regardless of computer speed.

## 3.4 How mecanum motion is simulated

A real mecanum wheel has rollers at 45° around its rim: it grips along one diagonal and rolls freely along the other. Simulating dozens of rollers is slow and numerically fragile, so each wheel is a cylinder with **anisotropic friction**: `mu = 1` along the diagonal `fdir1` and `mu2 = 0` across it. The four diagonals form an X seen from above. Spinning the wheels in different combinations then pushes the body forward, sideways, diagonally or rotates it, exactly like real mecanum wheels. The direction is expressed in `base_footprint` so it does not spin with the wheel.

## 3.5 How a nudge is applied

The ApplyLinkWrench system applies a force/torque at a link's origin. The pendulum link's origin is the pivot, and a force at the pivot cannot tip the pole, so the node converts the finger force F at height h into the torque it produces about the pivot, τ = F·h, about the horizontal axis perpendicular

to the push. Persistent wrenches stay applied until cleared and Gazebo sums successive ones, so the node keeps track of the torque currently applied and sends only the difference every 10 ms. This was verified in a test where sending the same wrench twice gave a much larger angular acceleration.

## 3.6 Why some tests were isolated

Gazebo processes find each other through `GZ_PARTITION` and ROS nodes through `ROS_DOMAIN_ID`. Running tests with unique values for both meant they could run in parallel and never touched your simulation. One test result was invalidated because two test runs accidentally shared a domain; it was re-run on an unused one.

# Part 4 — Control theory

## 4.1 The pendulum model

For one tilt axis, the pole pivots on a base that accelerates horizontally with acceleration $a$. Taking moments about the pivot (J = moment of inertia about the pivot, m = pendulum mass, l = distance from pivot to centre of mass, θ = tilt from vertical):

$$J \cdot \theta'' = m\,g\,l \cdot \sin\theta - m\,l \cdot a \cdot \cos\theta$$

Gravity tips the pole over (first term); accelerating the base under it pushes it back (second term). For small angles $\sin\theta \approx \theta$ and $\cos\theta \approx 1$:

$$\theta'' = \alpha\,\theta - \beta\,a, \quad \alpha = m\,g\,l / J, \quad \beta = m\,l / J$$

| Quantity | Value | From |
|---|---|---|
| J (about pivot) | 0.0334 kg·m$^2$ | rod mL$^2$/3 + ball mL$^2$ + 2mr$^2$/5 |
| m l | 0.075 kg·m | 0.1·0.25 + 0.1·0.5 |
| α | 22.06 s$^{-2}$ | gravity strength |
| β | 2.248 m$^{-1}$ | control authority |
| Fall time constant 1/√α | 0.213 s | how fast an uncontrolled tilt grows (×e every 0.21 s) |

## 4.2 Why the first draft (PD tilt → velocity) could not work

The base follows velocity commands, so commanding $v = K_p\theta + K_d\theta'$ means the base acceleration is its derivative, $a = K_p\theta' + K_d\theta''$. Substituting into the model:

$$(1 + \beta K_d)\,\theta'' + \beta K_p\,\theta' - \alpha\,\theta = 0$$

The constant term ($-\alpha\theta$) is negative for any gains, which by the Routh criterion always leaves an unstable root: the pole falls whatever $K_p$ and $K_d$ are. The fix is to command an acceleration that is proportional to θ (so the constant term becomes $\beta K\theta - \alpha > 0$) and integrate it into velocity.

## 4.3 State feedback with LQR

The state per axis is x = [θ, θ', p, v] (tilt, tilt rate, base position, base velocity) with input a:

$$\theta'' = \alpha\theta - \beta a, \quad p' = v, \quad v' = a$$

LQR (Linear Quadratic Regulator) finds the gains K in $a = k_\theta\theta + k_\omega\theta' + k_p p + k_v v$ that minimise $\int(x^T Q x + Ra^2)dt$: a trade-off between keeping the state small (Q) and using little acceleration (R). It was solved with `scipy.linalg.solve_continuous_are`.

| Design | Q | R | $k_\theta$ | $k_\omega$ | $k_p$ | $k_v$ | Closed-loop poles |
|---|---|---|---|---|---|---|---|
| Stiff (step 2) | diag(60,1,2,3) | 0.1 | 60.0 | 12.1 | 4.5 | 8.7 | −7.98±1.74j, −1.6, −0.92 |
| **Soft (current)** | diag(20,1,0.5,1) | 0.3 | **33.4** | **7.1** | **1.3** | **3.1** | −6.5, −4.49, −0.98±0.09j |

Note the positive sign of $k_p$ and $k_v$: to move the base back, the controller first moves it the *other* way to tip the pole toward home. This is the non-minimum-phase behaviour of every inverted pendulum and the reason braking needs a backward lean. The fastest pole cannot be much slower than $\sqrt{\alpha} \approx 4.7\ s^{-1}$, which is why the controller cannot be made arbitrarily “lazy”.

## 4.4 Why the 2D problem splits into two 1D problems

The pole hangs from two perpendicular hinges (pitch about Y, roll about X). For small angles, pitch is driven only by base acceleration along X and roll only by acceleration along Y, and the mecanum base can accelerate along X and Y independently. So one identical controller runs per axis. Signs: positive pitch tips toward +X; positive roll tips toward −Y, so the Y controller works in negated coordinates.

## 4.5 The cascade form and the limits

The four-term LQR law can be rewritten exactly as two levels:

$$\text{lean}_{\text{ref}} = -(k_p \cdot e_p + k_v \cdot e_v) / k_\theta \qquad a = k_\theta(\theta - \text{lean}_{\text{ref}}) + k_\omega \theta'$$

Without limits this is identical to the original law. The benefit is that each part can be limited for a physical reason:

| Limit | Value | Why |
|---|---|---|
| $\lvert\text{lean}_{\text{ref}}\rvert \le$ max_lean_rad | 0.06 rad (3.4°) | Braking/returning may never tip the pole far the other way (fixes the overshoot). |
| $\lvert k_p e_p\rvert \le k_v \cdot$max_return_speed | 0.3 m/s | The drive back home is gentle however far a nudge carried the robot. |
| $\lvert a\rvert \le$ max_accel (2D) | 6 $m/s^2$ | Never ask for more than the wheels can deliver, so the commanded velocity stays equal to the real one. |
| $\lvert v\rvert \le$ max_vel (2D) | 2 m/s | Absolute speed limit; limited value written back to avoid wind-up. |
| engage_tilt_rad | 0.17 rad (10°) | Visible free fall at start; largest catchable angle. |
| max_tilt_rad | 0.6 rad (34°) | Beyond this the pole is lost; stop the robot. |

**Why integrate acceleration instead of using measured velocity?** On a real robot and in the simulation, the base's velocity controller follows commands closely as long as the acceleration is achievable. Integrating the commanded acceleration gives a smooth, noise-free velocity estimate. The traction measurements (4.6) are what made this valid: once the acceleration limit matched the wheels, the commanded and real velocities agreed.

## 4.6 Measured results

### Wheel traction (velocity lag while accelerating to 1 m/s, ground truth)

| Direction | 2 m/s$^2$ | 5 m/s$^2$ | 8 m/s$^2$ |
|---|---|---|---|
| Forward | 0.06 m/s | 0.12 m/s | 0.31 m/s |
| Sideways | 0.03 m/s | 0.08 m/s | 0.25 m/s |
| Diagonal | 0.03 m/s | 0.09 m/s | 0.42 m/s |

### Acceleration limit vs. strongest survivable nudge (at the top of the pole)

| max_accel | Straight | Diagonal |
|---|---|---|
| 8 m/s$^2$ per axis (old) | 1.5 N ok | 1.5 N falls |
| 4 m/s$^2$ (2D) | 1.0 N ok, 1.5 N falls | 1.0 N ok |
| 5 m/s$^2$ (2D) | 1.5 N ok, 1.75 N falls | 1.5 N ok |
| **6 m/s$^2$ (2D, chosen)** | **1.75 N ok** | **1.5 N ok**, 1.75 N falls |

### Overshoot before and after the lean limit

| Nudge | Opposite lean before | Opposite lean after | Peak speed after | Travel after |
|---|---|---|---|---|
| 0.8 N | 4.4° | 3.9–4.4° | 0.56–0.67 m/s | 0.4–1.1 m |
| 1.2 N | 7.0–7.5° | 4.8° | 1.09–1.32 m/s | 1.0–1.2 m |
| 1.5 N | 9.6–12.0° | 4.8° | 1.43–1.73 m/s | 1.7–1.8 m |
| 1.75 N straight | 13.0° | 4.8° | 1.66 m/s | 2.2 m |

Lean caps of 2.0°, 2.9° and 4.6° were also compared: 2.0° could not brake enough and ran away; 4.6° allowed 6.4° opposite lean with shorter travel.

## 4.7 Gravity check: free fall vs. theory

| From 1° to | 2° | 5° | 10° | 20° | 45° | 90° |
|---|---|---|---|---|---|---|
| Simulation | 0.107 s | 0.286 s | 0.431 s | 0.578 s | 0.750 s | 0.881 s |
| Theory (fixed pivot) | 0.114 s | 0.296 s | 0.442 s | 0.589 s | 0.764 s | 0.920 s |

The simulation is 2–4% faster because the robot base recoils slightly as the pole falls (theory assumes an immovable pivot). Gravity and the pendulum's mass properties are therefore modelled correctly.

## 4.8 Design decisions and alternatives

| Decision | Alternative considered | Reason for the choice |
|---|---|---|
| Velocity commands via MecanumDrive | Torque control of each wheel | Matches the real LIMO's interface (it takes /cmd_vel), simpler, and the plugin handles mecanum kinematics. |
| Anisotropic friction wheels | Modelling every roller; or Gazebo's VelocityControl (moves the body directly) | Rollers are slow and unstable; VelocityControl would ignore wheels and traction entirely, hiding the very limit found in Step 7. |
| LQR, written as a cascade | PID per axis | LQR handles the coupled 4-state system with gains from the physics; the cascade adds physically meaningful limits without losing that. |
| Integrated commanded velocity as the estimate | Wheel odometry | Smooth and delay-free; wheel odometry cannot see slip either. |
| Torque at the pivot for nudges | Force with offset | The ROS EntityWrench message has no force offset; torque = F·h is physically equivalent for tipping the pole. |
| Sending wrench changes | Clear then set each time | Clear and set travel on different topics and can arrive out of order; deltas are exact. |
| Single-process Gazebo | Separate server and GUI | A GUI attaches to any server; one process guarantees the right world. |
| Simplified meshes | Using STL or the original DAE | The original froze the GUI; the STL has no colours. |
| Both Gazebo-window and terminal keyboard input | Only one | The window needs focus on the 3D view; the terminal works everywhere. One shared topic keeps one code path. |

# Part 5 — Running, tuning and reference

## 5.1 Build and run

```
source /opt/ros/jazzy/setup.bash
cd ~/omni_pendulum_ws
colcon build --symlink-install          # only after adding new files/programs
source install/setup.bash
ros2 launch omni_pendulum_sim sim.launch.py
```

Launch options: `gui:=false` (no window), `push:=false` (no nudges), `balance:=false` (no controller: watch the pole fall).

## 5.2 Nudging

- **Gazebo window:** click the 3D view, then ↑/↓ forward/back, ←/→ left/right, +/- strength (starts at 0.8 N, steps of 0.2 N).
- **Terminal:** `ros2 run omni_pendulum_sim keyboard_nudge` (same keys, q to quit).
- **Command:** `ros2 topic pub --once /nudge geometry_msgs/msg/Vector3 "{x: 0.5, y: 0.0}"`

## 5.3 Parameters

| Node | Parameter | Default | Effect |
|---|---|---|---|
| balance_controller | k_theta, k_omega, k_pos, k_vel | 33.4, 7.1, 1.3, 3.1 | LQR gains |
| | max_lean_rad | 0.06 | Higher: shorter roll-out, more opposite lean. Keep ≥ 0.05. |
| | max_return_speed | 0.3 m/s | Speed of the drive back home |
| | max_accel | 6.0 $m/s^2$ | Must match what the wheels can deliver |
| | max_vel | 2.0 m/s | Absolute speed limit |
| | engage_tilt_rad | 0.17 | Free-fall angle before catching (≤ ~0.2) |
| | max_tilt_rad | 0.6 | Give-up angle |
| | control_period | 0.005 s | Control rate (200 Hz) |
| | Override example: `ros2 run omni_pendulum_sim balance_controller --ros-args -p use_sim_time:=true -p max_lean_rad:=0.08` | | |
| disturbance | auto_nudge | True | Random nudges on/off |
| | min/max_interval | 4 / 8 s | Time between random nudges |
| | min/max_force | 0.3 / 1.0 N | Random nudge strength |
| | nudge_duration | 0.15 s | Length of a tap |
| | min/max_contact_height | 0.3 / 0.5 m | Where on the rod the tap lands |
| | key_force, key_force_step | 0.8, 0.2 N | Arrow-key strength |

## 5.4 Troubleshooting

| Symptom | Cause / fix |
|---|---|
| Package 'omni_pendulum_sim' not found | `source ~/omni_pendulum_ws/install/setup.bash` in that terminal. |
| Robot missing / two simulations / “Moved backwards in time” | A leftover Gazebo is running: `pkill -f "gz sim"`, then relaunch. |
| Arrow keys do nothing in Gazebo | Click on the 3D view first, or use keyboard_nudge in a terminal. |
| Gazebo “not responding” | Wait; if it persists, force Gazebo onto the NVIDIA GPU (laptops with hybrid graphics). |
| Pole falls after a very strong push | Expected above about 1.5–1.75 N at the top of the pole: the physical limit. |

## 5.5 Supporting files (not explained line by line)

- **worlds/balance_world.sdf** — a 1 ms physics step at real-time factor 1; the Physics, UserCommands and SceneBroadcaster systems (standard for any Gazebo world); **ApplyLinkWrench** (lets the disturbance node and the GUI push links); a sun light and a 100 m ground plane.
- **config/gui.config** — a copy of Gazebo Harmonic's default GUI layout with two changes: the camera pose `-1.6 0 1.1 0 0.39 0` (1.6 m behind, 1.1 m up, looking along +X so screen-up is robot-forward) and the **KeyPublisher** plugin appended.
- **package.xml** — package metadata and runtime dependencies (rclpy, message packages, launch, xacro, robot_state_publisher, ros_gz_sim, ros_gz_bridge, ros_gz_interfaces).
- **meshes/limo/** — the original AgileX meshes and the simplified `_lite` versions. The AgileX repository declares no licence; check with AgileX before publishing these files.

## 5.6 Moving to a real LIMO

- Measure the real robot's achievable acceleration and set `max_accel` to it; the controller relies on the base following its commands.
- Add a pendulum angle sensor (two encoders at the hinges or an IMU on the rod) and publish it as `/joint_states` with the same joint names, including angular velocity.
- Re-run the LQR design with the real pole's mass, length and inertia (Part 4.1–4.3), and expect sensor noise and latency to reduce the recoverable nudge strength.

# Part 6 — Line-by-line code walkthrough

Each source line is shown with its line number. The grey box under a line (or under a group of lines) explains what it does and why. Blank lines are shown but not explained; docstrings and closing brackets are explained as a group.

# balance_controller.py

`src/omni_pendulum_sim/omni_pendulum_sim/balance_controller.py` · 208 lines

The brain of the robot: reads the pendulum angles and decides how the base moves.

```
"""
Balance a 2-DOF inverted pendulum on a mecanum (omnidirectional) AgileX LIMO.

Each tilt axis is treated as an independent cart-pole:
  pitch (tilt toward +X)  -> drive the base along X
  roll  (tilt toward -Y)  -> strafe the base along Y
so whichever way the pole starts to fall, the robot drives in that direction to
get back underneath it.

The base only accepts velocity commands (/cmd_vel), but a pendulum is balanced
by *accelerating* under it, so the controller computes a desired acceleration
and integrates it into a velocity command. It is an LQR state feedback written
as a cascade, so the parts can be limited separately:

  lean_ref = -(k_pos * (p - p_ref) + k_vel * (v - v_ref)) / k_theta
  a = k_theta * (theta - lean_ref) + k_omega * theta_dot
  v += a * dt,  p += v * dt

The inner line keeps the pole up and is never limited. The outer line is how
the robot slows down and returns home: to brake it has to tip the pole slightly
backwards first (an inverted pendulum cannot be stopped any other way). Capping
lean_ref at max_lean_rad stops the robot from braking so hard after a big
nudge that the pole swings over to the other side and falls; capping the
position term at max_return_speed keeps the drive back home gentle.

At startup the controller stays idle and lets the pole fall freely until it
has tipped past engage_tilt_rad, then catches it.

The default gains come from a deliberately soft LQR design (so the pole visibly
sways while the robot works to keep it up) on the linearised model
  theta_ddot = (m g l / J) * theta - (m l / J) * a
for the pendulum in robot.urdf.xacro (rod 0.5 m / 0.1 kg + 0.1 kg ball on top).
"""

import math
```

**Lines 1–33** The module docstring: a plain-English summary of the whole controller, kept at the top of the file so anyone opening it understands the idea before reading code. It states the three key ideas: (1) each tilt axis is treated as its own cart-pole, (2) the base takes velocity commands, so the controller computes an acceleration and integrates it into a velocity, and (3) the control law is an LQR state feedback rewritten as a two-level cascade so that braking can be limited without ever limiting the part that keeps the pole up. It also records where the default gains came from (the linearised pendulum model).

Standard-library maths module. Used for `math.hypot` (length of a 2D vector) and `math.degrees` (radians to degrees for log messages).

```
36
37 from geometry_msgs.msg import Twist
```

`Twist` holds a linear and an angular velocity. It is the command format of Gazebo's MecanumDrive system: `linear.x` = forward speed, `linear.y` = sideways speed, both in m/s in the robot frame.

```
38 import rclpy
```

`rclpy` is the ROS 2 client library for Python. It provides `init`, `spin` and `shutdown`, which start the ROS middleware, process incoming messages and stop cleanly.

```
39 from rclpy.node import Node
```

`Node` is the base class of every ROS 2 program. Subclassing it gives this controller a name on the ROS graph, parameters, publishers, subscribers, timers and a logger.

```
40 from sensor_msgs.msg import JointState
```

`JointState` is the standard message carrying joint names, positions, velocities and efforts. Gazebo's JointStatePublisher system sends one for every joint of the robot, including the two pendulum joints; ros_gz_bridge converts it to this ROS type.

```
41
42
43 class AxisController:
```

One `AxisController` object handles one tilt direction. The same class is used twice (X and Y) because the pendulum is symmetric and a mecanum base can move along X and Y independently, so the two-dimensional problem splits into two identical one-dimensional cart-pole problems.

```
44     """LQR balancer for one tilt axis."""
```

Docstring for the class.

```
45
46     def __init__(self, k_theta, k_omega, k_pos, k_vel):
```

Constructor. It receives the four LQR gains that define how strongly the controller reacts to each part of the state.

```
47         self.k_theta = k_theta
```

`k_theta`: how many m/s$^2$ of base acceleration to command per radian of pole tilt. This is the main 'get underneath the pole' term.

```
48         self.k_omega = k_omega
```

`k_omega`: acceleration per rad/s of tilt rate. It reacts to how fast the pole is falling, which damps the motion and stops the base from oscillating around the pole.

```
49         self.k_pos = k_pos
```

`k_pos`: how strongly the robot is pulled back to its reference position (its starting point unless you drive it with /cmd_vel_ref).

```
50         self.k_vel = k_vel
```

`k_vel`: how strongly the robot's velocity is pulled toward the reference velocity (zero when standing). This is what makes it slow down after catching the pole.

```
51         self.reset()
```

Start from a clean state (zero velocity and position).

```
52
53     def reset(self):
```

`reset` clears the controller's internal memory. It is called at start-up and every time the pendulum has fallen, so the next attempt does not inherit a stale velocity.

```
54         self.vel = 0.0
```

`self.vel` is the velocity the controller has commanded so far. The base is velocity-controlled and was measured (with ground-truth odometry) to follow commands closely within the acceleration limit, so the commanded velocity doubles as the controller's estimate of the real base velocity.

```
55          self.pos = 0.0
```

`self.pos` is the integral of the commanded velocity: how far the controller believes the robot has travelled from where balancing started.

```
56          self.pos_ref = 0.0
```

`self.pos_ref` is where the robot should be. It stays at 0 unless someone publishes a desired velocity on /cmd_vel_ref, in which case it moves along at that velocity.

```
57
58      def lean_ref(self, vel_ref, max_return_speed, dt):
```

The outer loop. It answers: 'how much should the pole lean so that the robot slows down / goes home?' To decelerate, an inverted pendulum on a cart must first be tipped slightly backwards, so braking is expressed as a desired lean angle.

```
59          """Lean the pole should have to brake / return home (before limiting)."""
```

Docstring: the returned lean is the raw request, before the caller applies the `max_lean_rad` cap.

```
60          self.pos_ref += vel_ref * dt
```

Advance the reference position by the reference velocity times the time step (simple Euler integration). With no /cmd_vel_ref this adds zero.

```
61          pos_term = self.k_pos * (self.pos - self.pos_ref)
```

Position error term: gain times how far the robot is from where it should be.

```
62          pos_limit = self.k_vel * max_return_speed
```

Largest allowed position term. In steady state the outer loop balances `k_pos*error` against `k_vel*velocity`, so capping the position term at `k_vel*max_return_speed` means the robot can never drive home faster than `max_return_speed` (0.3 m/s), however far a nudge carried it.

```
63          pos_term = max(min(pos_term, pos_limit), -pos_limit)
```

Clamp the position term to plus/minus that limit.

```
64          return -(pos_term + self.k_vel * (self.vel - vel_ref)) / self.k_theta
```

The LQR control law rearranged: a lean of `-(k_pos*e_p + k_vel*e_v)/k_theta` produces exactly the same acceleration as the original four-term LQR law, but now the 'braking' part is a single number (an angle) that can be limited. The minus sign: if the robot is moving forward (positive velocity error), the pole must lean backwards (negative angle) so the robot decelerates.

```
65
66      def accel(self, theta, theta_dot, lean_ref):
```

The inner loop: the acceleration that keeps the pole up.

```
67          return self.k_theta * (theta - lean_ref) + self.k_omega * theta_dot
```

`a = k_theta*(theta - lean_ref) + k_omega*theta_dot`. If the pole tilts forward more than requested, accelerate forward under it; the `theta_dot` term reacts to the falling speed and adds damping. This part is never limited except by the physical acceleration limit, because limiting it would let the pole fall.

```
68
69      def integrate(self, accel, dt):
```

Turns the chosen acceleration into a new velocity and position.

```
70          self.vel += accel * dt
```

Euler integration: new velocity = old velocity + acceleration * time step. This is why the controller can command acceleration even though the base only accepts velocity.

```
71          self.pos += self.vel * dt
```

Integrate velocity into position, giving the position estimate used by the outer loop.

```
72
73
74 def limit_norm(x, y, limit):
```

Helper that limits the length of a 2D vector while keeping its direction. Used for the lean, acceleration and speed limits so that diagonal motions are scaled as a whole instead of being clipped per axis (per-axis clipping would bend a diagonal command toward an axis).

```
75     """Scale the vector (x, y) down to length `limit`, keeping its direction."""
```

Docstring.

```
76     norm = math.hypot(x, y)
```

Length of the vector, `sqrt(x*x + y*y)`.

```
77     if norm <= limit:
```

If it is already within the limit...

```
78         return x, y
```

...return it unchanged.

```
79     return x * limit / norm, y * limit / norm
```

Otherwise scale both components by `limit/norm`, which shortens the vector to exactly `limit` without changing its direction.

```
80
81
82 class OmnidirectionalBalanceController(Node):
```

The ROS 2 node class. It owns the two axis controllers, reads parameters, subscribes to sensor topics and publishes velocity commands.

```
83
84     def __init__(self):
```

Constructor, runs once when the node starts.

```
85         super().__init__('balance_controller')
```

Registers the node on the ROS graph under the name `balance_controller` (visible with `ros2 node list`).

```
86
87         self.declare_parameter('k_theta', 33.4)
```

Declares the ROS parameter `k_theta` with default 33.4. The four gains come from an LQR design with Q = diag(20, 1, 0.5, 1) and R = 0.3 on the linearised pendulum model. Declaring them as parameters means they can be changed at launch (`--ros-args -p k_theta:=...`) without editing code.

```
88         self.declare_parameter('k_omega', 7.1)
```

Tilt-rate gain `k_omega` = 7.1.

```
89         self.declare_parameter('k_pos', 1.3)
```

Position gain `k_pos` = 1.3.

```
90         self.declare_parameter('k_vel', 3.1)
```

Velocity gain `k_vel` = 3.1.

```
91         # Hard safety limit; normally the lean limit keeps the robot well below it
```

Comment explaining the next parameter.

```
92         self.declare_parameter('max_vel', 2.0)
```

`max_vel` = 2.0 m/s: an absolute speed limit. The robot never goes faster, even if the controller asks.

```
93         # How far the controller may deliberately tip the pole to brake or
94         # return home, and how fast it drives back to where it started
```

**Lines 93–94** Comment explaining the next two parameters.

```
95          self.declare_parameter('max_lean_rad', 0.06)
```

`max_lean_rad` = 0.06 rad (3.4 degrees): the most the controller may deliberately tip the pole when braking or returning home. This is the fix for the 'robot runs too fast and the pole leans to the other side' problem: tests showed the opposite lean stays under 5 degrees for every survivable nudge. Below 0.05 rad the robot cannot brake hard enough and runs away.

```
96          self.declare_parameter('max_return_speed', 0.3)
```

`max_return_speed` = 0.3 m/s: how fast the robot drives back to its starting point after a nudge.

```
97          # What the mecanum wheels can actually deliver in any direction (the
98          # diagonal is the weakest). The controller assumes the base follows its
99          # commands, so asking for more than this makes it lose track and overshoot.
```

**Lines 97–99** Comment explaining why the acceleration limit exists.

```
100         self.declare_parameter('max_accel', 6.0)
```

`max_accel` = 6.0 m/s$^2$: the most acceleration the controller will ask for. Tests with ground-truth odometry showed the mecanum wheels cannot accelerate the robot much harder (especially diagonally). When the controller asked for 8 m/s$^2$, the real robot lagged behind, the controller's velocity estimate became wrong, it braked too late and dropped the pole. 4, 5, 6 and 8 were compared and 6 gave the best nudge tolerance.

```
101         self.declare_parameter('max_tilt_rad', 0.6)
```

`max_tilt_rad` = 0.6 rad (34 degrees): beyond this the pole cannot be saved, so the controller gives up and stops the base instead of chasing a fallen pole across the floor.

```
102         self.declare_parameter('engage_tilt_rad', 0.17)
```

`engage_tilt_rad` = 0.17 rad (10 degrees): the controller stays idle at start-up until the pole has fallen this far, so you can see gravity act on it first. 10 degrees is close to the largest angle from which the robot can still catch it (15 degrees failed in tests).

```
103         self.declare_parameter('control_period', 0.005)
```

`control_period` = 0.005 s: run the control law at most every 5 ms (200 Hz). Joint states arrive about every millisecond; 200 Hz is far faster than the pendulum dynamics (time constant about 0.2 s) and keeps CPU load low.

```
104
105         gains = [self.get_parameter(n).value for n in ('k_theta', 'k_omega', 'k_pos', 'k_vel')]
```

Read the four gain parameters into a list (their values may have been overridden at launch).

```
106         self.x_axis = AxisController(*gains)
```

Create the X-axis controller (pitch tilt, forward/back motion).

```
107         self.y_axis = AxisController(*gains)
```

Create the Y-axis controller (roll tilt, sideways motion) with the same gains.

```
108         self.max_vel = self.get_parameter('max_vel').value
109         self.max_accel = self.get_parameter('max_accel').value
110         self.max_lean = self.get_parameter('max_lean_rad').value
111         self.max_return_speed = self.get_parameter('max_return_speed').value
112         self.max_tilt_rad = self.get_parameter('max_tilt_rad').value
113         self.engage_tilt_rad = self.get_parameter('engage_tilt_rad').value
114         self.control_period = self.get_parameter('control_period').value
```

**Lines 108–114** Read the remaining parameters into attributes so they are fast to access inside the control loop.

```
115
116         # Optional user velocity request (e.g. from teleop_twist_keyboard)
```

Comment: the next two attributes allow an external program to request a travel velocity while balancing.

```
117         self.vel_ref_x = 0.0
```

Requested forward velocity from /cmd_vel_ref (0 by default).

```
        self.vel_ref_y = 0.0
```

Requested sideways velocity from /cmd_vel_ref.

```
        self.last_stamp = None
```

Simulation time of the last control step; `None` until the first message arrives.

```
        self.engaged = False
```

`engaged` becomes True once the pole has passed the engage angle and balancing has started.

```
        self.fallen = False
```

`fallen` remembers whether the pole is currently past the safety limit, so the warning is logged only once per fall.

```

        self.create_subscription(JointState, '/joint_states', self.joint_callback, 10)
```

Subscribe to /joint_states. Every incoming message calls `joint_callback`. The `10` is the queue depth: up to 10 messages are buffered if the callback is busy.

```
        self.create_subscription(Twist, '/cmd_vel_ref', self.ref_callback, 10)
```

Subscribe to /cmd_vel_ref, an optional desired travel velocity (for example from teleop_twist_keyboard remapped to this topic).

```
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
```

Create the publisher for /cmd_vel. ros_gz_bridge forwards it to Gazebo's MecanumDrive system, which turns it into wheel speeds.

```

        self.get_logger().info('Balance controller started')
```

Log a start-up message to the terminal.

```

    def ref_callback(self, msg):
```

Callback for /cmd_vel_ref messages.

```
        self.vel_ref_x = msg.linear.x
```

Store the requested forward velocity.

```
        self.vel_ref_y = msg.linear.y
```

Store the requested sideways velocity.

```

    def joint_callback(self, msg):
```

The main control loop. It runs every time Gazebo publishes joint states (about 1000 times per second of simulated time) and decides the next velocity command.

```
        try:
```

The joint order in the message is not guaranteed, so look the pendulum joints up by name.

```
            pitch_idx = msg.name.index('pendulum_pitch_joint')
```

Index of the pitch joint (tilt about the robot's Y axis, i.e. leaning forward/back).

```
            roll_idx = msg.name.index('pendulum_roll_joint')
```

Index of the roll joint (tilt about the robot's X axis, i.e. leaning left/right).

```
        except ValueError:
```

If a joint is missing (for example a message published before the robot was fully spawned)...

```
            return
```

...ignore this message.

```

        theta_pitch = msg.position[pitch_idx]
```

Pitch angle in radians. Positive means the top of the pole leans toward +X (forward).

```
        theta_roll = msg.position[roll_idx]
```

Roll angle in radians. Positive means the top of the pole leans toward -Y (right).

```
        omega_pitch = msg.velocity[pitch_idx] if msg.velocity else 0.0
```

Pitch angular velocity in rad/s, measured directly by Gazebo (no noisy numerical differentiation needed). Falls back to 0 if the message carries no velocities.

```
        omega_roll = msg.velocity[roll_idx] if msg.velocity else 0.0
```

Roll angular velocity in rad/s.

```

        # Use the message's simulation timestamp so dt matches simulated physics
```

Comment: why the message timestamp is used.

```
        stamp = msg.header.stamp.sec + msg.header.stamp.nanosec * 1e-9
```

Convert the message's header stamp (seconds + nanoseconds of simulation time) to seconds. Using simulation time means dt is correct even when Gazebo runs slower or faster than real time.

```
        if self.last_stamp is None:
```

On the very first message there is no previous time...

```
            self.last_stamp = stamp
```

...so remember this time...

```
            return
```

...and wait for the next message.

```
        dt = stamp - self.last_stamp
```

Time elapsed since the last control step.

```
        if dt < self.control_period:
```

If less than `control_period` has passed...

```
            return
```

...skip this message (rate limiting to 200 Hz).

```
        self.last_stamp = stamp
```

Remember the time of this control step.

```

        cmd = Twist()
```

Create an all-zero velocity command. It is filled in later, or published as-is to stop the robot.

```

        # Let the pole fall freely until it has tipped far enough, then catch it
```

Comment: free-fall phase at start-up.

```
        if not self.engaged:
```

While balancing has not started yet...

```
            if max(abs(theta_pitch), abs(theta_roll)) < self.engage_tilt_rad:
```

...check whether the pole is still within the engage angle on both axes...

```
                self.pub.publish(cmd)
```

...if so, keep the robot still (publish zero velocity)...

```
                return
```

...and do nothing else, letting gravity tip the pole.

```
            self.engaged = True
```

The pole has fallen past the engage angle: start balancing from now on.

```
            self.get_logger().info(
                f'Pole tipped past {math.degrees(self.engage_tilt_rad):.0f} deg, balancing')
```

**Lines 163–164** Log the moment balancing starts, e.g. 'Pole tipped past 10 deg, balancing'.

```
165
166         # Safety cutoff once the pendulum has fallen past recovery
```

Comment: safety cut-off.

```
167         if abs(theta_pitch) > self.max_tilt_rad or abs(theta_roll) > self.max_tilt_rad:
```

If either angle exceeds the recoverable limit (34 degrees)...

```
168             if not self.fallen:
```

...and this is the first control step since it fell...

```
169                 self.get_logger().warn('Pendulum fell over, stopping base')
```

...log a warning once...

```
170                 self.fallen = True
```

...and remember that it is down.

```
171             self.x_axis.reset()
```

Clear the X controller's memory.

```
172             self.y_axis.reset()
```

Clear the Y controller's memory.

```
173             self.pub.publish(cmd)
```

Stop the robot.

```
174             return
```

Skip the control law.

```
175         self.fallen = False
```

The pole is within limits, so clear the fallen flag.

```
176
177         # Positive pitch tips toward +X; positive roll tips toward -Y, so the
178         # y axis works in negated coordinates. Limits are applied to the 2D
179         # vectors so diagonal moves keep their direction.
```

**Lines 177–179** Comment explaining the sign convention and the 2D limits used below.

```
180         lean_x = self.x_axis.lean_ref(self.vel_ref_x, self.max_return_speed, dt)
```

Outer loop for X: the lean needed to brake/return along X.

```
181         lean_y = self.y_axis.lean_ref(-self.vel_ref_y, self.max_return_speed, dt)
```

Outer loop for Y. The Y axis works in negated coordinates (because positive roll tips toward -Y), so the requested Y velocity is negated on the way in.

```
182         lean_x, lean_y = limit_norm(lean_x, lean_y, self.max_lean)
```

Cap the combined requested lean at `max_lean_rad`. This is the key anti-overshoot limit.

```
183         ax = self.x_axis.accel(theta_pitch, omega_pitch, lean_x)
```

Inner loop for X: acceleration from pitch angle, pitch rate and the requested lean.

```
184         ay = -self.y_axis.accel(theta_roll, omega_roll, lean_y)
```

Inner loop for Y, negated back into robot coordinates (+Y = left).

```
185         ax, ay = limit_norm(ax, ay, self.max_accel)
```

Cap the combined acceleration at what the wheels can deliver (`max_accel`), keeping the direction.

```
186         self.x_axis.integrate(ax, dt)
```

Integrate the X acceleration into the X velocity and position.

```
187         self.y_axis.integrate(-ay, dt)
```

Integrate the Y acceleration (negated again into the controller's internal Y convention).

```
188
189         vx, vy = limit_norm(self.x_axis.vel, -self.y_axis.vel, self.max_vel)
```

Cap the combined speed at `max_vel`, keeping the direction.

```
        self.x_axis.vel, self.y_axis.vel = vx, -vy
```

Write the limited velocities back into the controllers. Without this, the internal velocity would keep growing beyond what is actually commanded (integrator wind-up) and the controller would lose track of the real robot.

```
        cmd.linear.x = vx
```

Forward velocity command.

```
        cmd.linear.y = vy
```

Sideways velocity command.

```
        self.pub.publish(cmd)
```

Send the command to /cmd_vel.

```


def main(args=None):
```

`main` is the entry point that `ros2 run omni_pendulum_sim balance_controller` calls (registered in setup.py).

```
    rclpy.init(args=args)
```

Initialise ROS 2 for this process.

```
    node = OmnidirectionalBalanceController()
```

Create the controller node (runs `__init__`).

```
    try:
```

Start a block that handles Ctrl+C cleanly.

```
        rclpy.spin(node)
```

`spin` hands control to ROS: it waits for messages and calls the callbacks until shutdown.

```
    except KeyboardInterrupt:
```

Ctrl+C raises KeyboardInterrupt...

```
        pass
```

...which is expected, so it is ignored instead of printing a traceback.

```
    node.destroy_node()
```

Release the node's publishers and subscriptions.

```
    rclpy.try_shutdown()
```

Shut ROS down. `try_shutdown` does nothing if the launch system has already shut it down, avoiding an error on Ctrl+C.

```


if __name__ == '__main__':
```

Standard Python guard: only call `main` when this file is run directly...

```
    main()
```

...not when it is imported by another module.

# disturbance.py

`src/omni_pendulum_sim/omni_pendulum_sim/disturbance.py` · 177 lines

Simulates a person nudging the pole: automatic random taps, arrow keys and /nudge.

```
"""
Nudge the pendulum like a person tapping it with a finger.

A nudge is a short horizontal push on the pole: the force rises and falls as a
half-sine over nudge_duration seconds and acts at a point somewhere along the
upper part of the rod. Gazebo's ApplyLinkWrench system (bridged by
ros_gz_bridge) only takes a wrench at the link origin, which is the pivot, so
the push is applied as the equivalent torque: force * contact height.

Nudges happen:
  * once at startup (a very small one) so the pole starts to fall on its own
  * automatically, at random intervals, with random strength and direction
  * from the arrow keys, either in the Gazebo window (Key Publisher plugin) or
    in a terminal running keyboard_nudge; both publish Qt key codes on
    /keyboard/keypress. Up/Down push along +X/-X (robot forward/back),
    Left/Right along +Y/-Y, and +/- change the push strength
  * on demand, by publishing the push force (N) to /nudge, e.g.
      ros2 topic pub --once /nudge geometry_msgs/msg/Vector3 "{x: 0.5, y: 0.0}"

Gazebo adds persistent wrenches together rather than replacing them, so this
node only ever publishes the *change* in total torque since the last update.
"""
```

**Lines 1–22** Module docstring. It explains the physical model of a nudge (a half-sine force pulse at some height on the rod), why it is applied as a torque (Gazebo's ApplyLinkWrench can only apply a wrench at the link origin, which is the pivot, so a force F at height h becomes the equivalent torque F*h), the four ways a nudge can be triggered, and the important implementation detail that Gazebo adds persistent wrenches together, so the node always sends only the change in torque.

```
import math
```

Maths module: `sin`, `cos`, `atan2`, `hypot`, `pi`, `inf`, `degrees`.

```
import random
```

Random numbers for random nudge timing, strength, direction and contact height.

```
from geometry_msgs.msg import Vector3
```

`Vector3` (x, y, z) message, used on /nudge for a manual push force in newtons.

```
import rclpy
```

ROS 2 Python client library.

```
from rclpy.node import Node
```

Base class for ROS 2 nodes.

```
from ros_gz_interfaces.msg import Entity, EntityWrench
```

`Entity` identifies something in the Gazebo world (here: the pendulum link); `EntityWrench` is a force + torque to apply to an entity. Both come from ros_gz_interfaces and are bridged to the matching Gazebo messages.

```
from std_msgs.msg import Int32
```

`Int32` message, used for key codes on /keyboard/keypress.

```

# Qt key codes, as published by Gazebo's Key Publisher GUI plugin
```

Comment: the next constants are Qt key codes.

```
KEY_LEFT, KEY_UP, KEY_RIGHT, KEY_DOWN = 0x01000012, 0x01000013, 0x01000014, 0x01000015
```

Qt key codes for the arrow keys. Gazebo's GUI is built with Qt, and its Key Publisher plugin publishes these numbers when a key is pressed. keyboard_nudge.py publishes the same numbers, so both input methods share one code path.

```
KEY_DIRECTIONS = {KEY_UP: (1.0, 0.0), KEY_DOWN: (-1.0, 0.0),
                  KEY_LEFT: (0.0, 1.0), KEY_RIGHT: (0.0, -1.0)}
```

**Lines 35–36** Maps each arrow key to a unit push direction in the robot frame: Up = +X (forward), Down = -X, Left = +Y, Right = -Y. The default Gazebo camera looks along +X, so screen-up matches robot-forward.

```
KEY_STRONGER = (ord('+'), ord('='))
```

Key codes that increase the push strength: '+' and '=' (the same key without Shift).

```
KEY_WEAKER = (ord('-'), ord('_'))
```

Key codes that decrease the push strength: '-' and '_'.

```


class Disturbance(Node):
```

The disturbance node class.

```

    def __init__(self):
```

Constructor.

```
        super().__init__('disturbance')
```

Register the node as `disturbance`.

```

        self.declare_parameter('world', 'balance_world')
```

`world`: name of the Gazebo world, needed to build the wrench topic names.

```
        self.declare_parameter('link', 'omni_pendulum::pendulum_link')
```

`link`: the scoped Gazebo name of the link to push, `model::link`.

```
        self.declare_parameter('auto_nudge', True)
```

`auto_nudge`: whether random nudges happen automatically (True) or only on command (False).

```
        self.declare_parameter('release_delay', 4.0)
```

`release_delay`: seconds after start before the tiny release nudge, giving the robot time to spawn and settle.

```
        self.declare_parameter('release_force', 0.1)
```

`release_force`: 0.1 N, just enough to start the pole falling under gravity.

```
        self.declare_parameter('min_interval', 4.0)
```

`min_interval`: shortest random time between automatic nudges (s).

```
        self.declare_parameter('max_interval', 8.0)
```

`max_interval`: longest random time between automatic nudges (s).

```
        self.declare_parameter('min_force', 0.3)
```

`min_force`: weakest automatic nudge (N).

```
54          self.declare_parameter('max_force', 1.0)
```

`max_force`: strongest automatic nudge (N). 1.0 N leaves a safety margin below the measured recovery limit (about 1.5 N in every direction at the top of the pole).

```
55          self.declare_parameter('nudge_duration', 0.15)
```

`nudge_duration`: 0.15 s, roughly the contact time of a finger tap.

```
56          self.declare_parameter('min_contact_height', 0.3)
```

`min_contact_height`: lowest point on the rod where a nudge lands (m above the pivot).

```
57          self.declare_parameter('max_contact_height', 0.5)
```

`max_contact_height`: highest point (the rod is 0.5 m long).

```
58          self.declare_parameter('key_force', 0.8)
```

`key_force`: starting strength of arrow-key nudges (N).

```
59          self.declare_parameter('key_force_step', 0.2)
```

`key_force_step`: how much one '+' or '-' press changes the strength (N).

```
60
61          world = self.get_parameter('world').value
62          self.link = self.get_parameter('link').value
63          self.auto_nudge = self.get_parameter('auto_nudge').value
64          self.release_delay = self.get_parameter('release_delay').value
65          self.release_force = self.get_parameter('release_force').value
66          self.interval = (self.get_parameter('min_interval').value,
67                           self.get_parameter('max_interval').value)
68          self.force_range = (self.get_parameter('min_force').value,
69                              self.get_parameter('max_force').value)
70          self.duration = self.get_parameter('nudge_duration').value
71          self.height_range = (self.get_parameter('min_contact_height').value,
72                               self.get_parameter('max_contact_height').value)
73          self.key_force = self.get_parameter('key_force').value
74          self.key_force_step = self.get_parameter('key_force_step').value
```

**Lines 61–74** Read all parameters into attributes. Ranges are stored as (min, max) tuples so they can be passed straight to `random.uniform(*range)`.

```
75
76          self.wrench_pub = self.create_publisher(
77              EntityWrench, f'/world/{world}/wrench/persistent', 10)
```

**Lines 76–77** Publisher for `/world/balance_world/wrench/persistent`. ros_gz_bridge forwards it to Gazebo's ApplyLinkWrench system, which applies the wrench every physics step until cleared. A persistent wrench is used because a one-shot wrench only lasts a single 1 ms step, too short to shape a 150 ms push.

```
78          self.clear_pub = self.create_publisher(
79              Entity, f'/world/{world}/wrench/clear', 10)
```

**Lines 78–79** Publisher for the matching `wrench/clear` topic, used once at start-up to remove any leftover wrench.

```
80          self.create_subscription(Vector3, '/nudge', self.nudge_callback, 10)
```

Subscribe to /nudge for manual pushes (force vector in N).

```
81          self.create_subscription(Int32, '/keyboard/keypress', self.key_callback, 10)
```

Subscribe to /keyboard/keypress for arrow-key pushes and strength changes.

```
82
83          self.start_time = None
```

Simulation time when the node first saw the clock.

```
84          self.released = False
```

Whether the start-up release nudge has happened.

```
85          self.next_nudge = math.inf
```

Time of the next automatic nudge. Infinity until the release nudge has happened.

```
86          self.nudge = None           # (start time, peak torque about x, about y)
```

The nudge currently in progress: (start time, peak torque about X, peak torque about Y), or None.

```
87          self.pending = []           # manual nudges waiting for the next tick
```

Queue of manual / keyboard nudges waiting to be applied.

```
88          self.applied = [0.0, 0.0]   # total torque Gazebo currently applies
```

The total torque Gazebo is currently applying. Needed because Gazebo adds persistent wrenches together: to go from torque A to torque B, the node must send B - A.

```
89
90          self.create_timer(0.01, self.tick)
```

Run `tick` every 0.01 s of simulation time (100 Hz), which samples the 0.15 s half-sine pulse 15 times.

```
91
92      def entity(self):
```

Helper that builds the `Entity` message identifying the pendulum link.

```
93          ent = Entity()
```

New empty Entity message.

```
94          ent.name = self.link
```

Name of the link to push.

```
95          ent.type = Entity.LINK
```

Tell Gazebo the name refers to a link (not a model, joint, etc.).

```
96          return ent
```

Return it.

```
97
98      def nudge_callback(self, msg):
```

Callback for /nudge.

```
99          self.pending.append((msg.x, msg.y))
```

Queue the requested force (x, y) in newtons.

```
100
101     def key_callback(self, msg):
```

Callback for /keyboard/keypress.

```
102         if msg.data in KEY_DIRECTIONS:
```

If the key is an arrow key...

```
103             # Holding a key auto-repeats; don't let presses pile up
```

Comment: holding an arrow key makes the operating system repeat it many times per second.

```
104             if len(self.pending) < 2:
```

Only queue the press if fewer than 2 are waiting, so holding a key cannot stack up dozens of pushes and knock the pole over.

```
105                 dx, dy = KEY_DIRECTIONS[msg.data]
```

Look up the unit direction for this arrow.

```
106                 self.pending.append((dx * self.key_force, dy * self.key_force))
```

Queue a push of `key_force` newtons in that direction.

```
107         elif msg.data in KEY_STRONGER + KEY_WEAKER:
```

Otherwise, if the key is '+', '=', '-' or '_'...

```
            step = self.key_force_step if msg.data in KEY_STRONGER else -self.key_force_step
```

...choose +step or -step...

```
            self.key_force = min(max(self.key_force + step, self.key_force_step), 3.0)
```

...and change the strength, clamped between one step (0.2 N) and 3.0 N.

```
            self.get_logger().info(f'Arrow-key nudge strength: {self.key_force:.1f} N')
```

Log the new strength so you can see it in the launch terminal.

```

    def start_nudge(self, t, fx, fy, label):
```

Starts a nudge with force (fx, fy) newtons at time t.

```
        height = random.uniform(*self.height_range)
```

Pick a random contact height on the rod.

```
        # A force along +X at height h tips the pole about +Y, and +Y about -X
```

Comment: how a force becomes a torque.

```
        self.nudge = (t, -fy * height, fx * height)
```

Torque = height x force. A push along +X at height h rotates the pole about +Y (torque_y = fx*h); a push along +Y rotates it about -X (torque_x = -fy*h). These are the peak torques; the pulse shape scales them over time.

```
        self.get_logger().info(
            f'{label}: {math.hypot(fx, fy):.2f} N toward '
            f'{math.degrees(math.atan2(fy, fx)):.0f} deg at {height:.2f} m up the pole')
```

**Lines 116–118** Log the nudge: strength, direction in degrees (0 = forward, 90 = left) and contact height.

```

    def random_force(self, magnitude):
```

Returns a random force vector of the given magnitude.

```
        heading = random.uniform(-math.pi, math.pi)
```

Random direction between -180 and +180 degrees.

```
        return magnitude * math.cos(heading), magnitude * math.sin(heading)
```

Convert direction and magnitude into x and y components.

```

    def tick(self):
```

Runs every 10 ms: decides when nudges start and sends torque updates.

```
        t = self.get_clock().now().nanoseconds * 1e-9
```

Current simulation time in seconds.

```
        if t == 0.0:
```

Before the first /clock message arrives the time reads 0...

```
            return  # /clock not received yet
```

...so do nothing yet.

```
        if self.start_time is None:
```

On the first tick with a valid clock...

```
            self.start_time = t
```

...remember the start time...

```
            self.clear_pub.publish(self.entity())
```

...clear any persistent wrench left over from a previous run of this node...

```
            return
```

...and wait for the next tick.

```
132
133         if self.nudge is None:
```
Only start a new nudge when none is in progress.
```
134             if not self.released and t - self.start_time >= self.release_delay:
```
If the release nudge has not happened and its delay has passed...
```
135                 self.released = True
```
...mark it done...
```
136                 self.start_nudge(t, *self.random_force(self.release_force), 'Release nudge')
```
...start a tiny nudge in a random direction...
```
137                 if self.auto_nudge:
```
...and if automatic nudges are on...
```
138                     self.next_nudge = t + random.uniform(*self.interval)
```
...schedule the first one at a random interval.
```
139             elif self.pending:
```
Otherwise, if a manual or keyboard nudge is waiting...
```
140                 self.start_nudge(t, *self.pending.pop(0), 'Manual nudge')
```
...start the oldest one.
```
141             elif t >= self.next_nudge:
```
Otherwise, if it is time for an automatic nudge...
```
142                 self.start_nudge(t, *self.random_force(random.uniform(*self.force_range)), 'Nudg
e')
```
...start one with a random strength in [min_force, max_force] and a random direction...
```
143                 self.next_nudge = t + random.uniform(*self.interval)
```
...and schedule the next.
```
144
145         target = [0.0, 0.0]
```
Default: no torque.
```
146         if self.nudge is not None:
```
If a nudge is in progress...
```
147             start, tx, ty = self.nudge
```
...unpack its start time and peak torques.
```
148             phase = (t - start) / self.duration
```
Progress through the pulse, from 0 (start) to 1 (end).
```
149             if phase >= 1.0:
```
If the pulse is over...
```
150                 self.nudge = None
```
...end the nudge (target stays 0).
```
151             else:
```
Otherwise...
```
152                 shape = math.sin(math.pi * phase)
```
...compute the half-sine shape: 0 at the start, 1 in the middle, 0 at the end. A smooth pulse feels like a finger tap and avoids an instantaneous jolt.
```
153                 target = [tx * shape, ty * shape]
```
Scale the peak torques by the shape.

```
154
155         delta = [tg - ap for tg, ap in zip(target, self.applied)]
```

The change needed to go from the torque Gazebo applies now to the target torque.

```
156         if abs(delta[0]) + abs(delta[1]) > 1e-6:
```

Only publish if it changed noticeably (avoids flooding the bridge with zero updates).

```
157             msg = EntityWrench()
```

New wrench message.

```
158             msg.entity = self.entity()
```

Which link to apply it to.

```
159             msg.wrench.torque.x = delta[0]
```

Torque change about the world X axis (N*m).

```
160             msg.wrench.torque.y = delta[1]
```

Torque change about the world Y axis (N*m).

```
161             self.wrench_pub.publish(msg)
```

Send it; Gazebo adds it to the persistent wrench.

```
162             self.applied = target
```

Remember the new total.

```
163
164
165 def main(args=None):
166     rclpy.init(args=args)
167     node = Disturbance()
168     try:
169         rclpy.spin(node)
170     except KeyboardInterrupt:
171         pass
172     node.destroy_node()
173     rclpy.try_shutdown()
174
175
176 if __name__ == '__main__':
177     main()
```

**Lines 165–177** Entry point, identical in structure to the controller's: initialise ROS, create the node, spin until Ctrl+C, then clean up.

# keyboard_nudge.py

`src/omni_pendulum_sim/omni_pendulum_sim/keyboard_nudge.py` · 85 lines

Optional terminal program that turns arrow-key presses into nudges.

```
"""
Nudge the pendulum from a terminal with the arrow keys.

Run in its own terminal while the simulation is running:
    ros2 run omni_pendulum_sim keyboard_nudge

Key presses are published as Qt key codes on /keyboard/keypress, the same
topic Gazebo's Key Publisher plugin uses, and the disturbance node turns them
into nudges.
"""
```

**Lines 1–10** Module docstring: how to run the node and how it fits with the rest of the system (it publishes the same key codes as Gazebo's Key Publisher, so the disturbance node handles both).

```

import os
```

`os.read` reads raw bytes from the terminal without Python's buffering.

```
import select
```

`select` waits until the terminal has input, with a timeout, so the loop never blocks forever.

```
import sys
```

`sys.stdin` is the terminal input.

```
import termios
```

`termios` saves and restores terminal settings.

```
import tty
```

`tty.setcbreak` switches the terminal so each key press is delivered immediately (without waiting for Enter) and is not echoed.

```

from omni_pendulum_sim.disturbance import KEY_DOWN, KEY_LEFT, KEY_RIGHT, KEY_UP
```

Reuse the Qt key-code constants from disturbance.py so both files always agree.

```
import rclpy
```

ROS 2 Python client library.

```
from rclpy.node import Node
```

Base class for ROS 2 nodes.

```
from std_msgs.msg import Int32
```

`Int32` message used for key codes.

```


ARROWS = {'A': KEY_UP, 'B': KEY_DOWN, 'C': KEY_RIGHT, 'D': KEY_LEFT}
```

Terminals send arrow keys as the escape sequence ESC [ A/B/C/D. This maps the final letter to the Qt code: A = Up, B = Down, C = Right, D = Left.

```
NAMES = {KEY_UP: 'forward +X', KEY_DOWN: 'back -X', KEY_LEFT: 'left +Y', KEY_RIGHT: 'right -Y'}
```

Human-readable direction names for the feedback messages.

```
26
27  HELP = """
28  Nudge the pendulum:
29     Up / Down     push forward / back
30     Left / Right  push left / right
31     + / -         stronger / weaker push
32     q             quit
33  """
```

**Lines 27–33** The help text printed when the node starts.

```
34
35
36  def read_key(timeout):
```

Reads one key press, waiting at most `timeout` seconds. Returns a Qt key code for arrows, the character for other keys, or None.

```
37      # Read raw bytes: Python's buffered stdin would swallow the rest of an
38      # arrow key's escape sequence and hide it from select()
```

**Lines 37–38** Comment: why raw bytes are read. Python's `sys.stdin.read(1)` reads the whole arrow-key sequence into an internal buffer, so a following `select` would report no more data and the arrow would be lost. This was a real bug found while testing through a pseudo-terminal.

```
39      fd = sys.stdin.fileno()
```

File descriptor (OS-level handle) of the terminal input.

```
40      if not select.select([fd], [], [], timeout)[0]:
```

Wait for input; if none arrives within the timeout...

```
41          return None
```

...return None.

```
42      data = os.read(fd, 8).decode(errors='ignore')
```

Read up to 8 raw bytes and decode them to text.

```
43      if data.startswith('\x1b'):  # arrow keys arrive as ESC [ A..D
```

An arrow key starts with the escape character (byte 27)...

```
44          while len(data) < 3 and select.select([fd], [], [], 0.05)[0]:
```

...if the rest of the sequence has not arrived yet, wait up to 50 ms for it...

```
45              data += os.read(fd, 8).decode(errors='ignore')
```

...and append it.

```
46          return ARROWS.get(data[2]) if data[1:2] == '[' and len(data) >= 3 else None
```

If it is ESC [ followed by a letter, return the matching arrow code; otherwise (a lone Esc, other sequences) return None.

```
47      return data[0] if data else None
```

A normal key: return its first character.

```
48
49
50  def main(args=None):
```

Entry point for `ros2 run omni_pendulum_sim keyboard_nudge`.

```
51      if not sys.stdin.isatty():
```

This node needs a real terminal to read keys from...

```
52          print('keyboard_nudge needs an interactive terminal; run it with ros2 run.')
```

...so explain the problem if it is started without one (e.g. from a launch file)...

```
        return
```

...and exit.

```

    rclpy.init(args=args)
```

Initialise ROS 2.

```
    node = Node('keyboard_nudge')
```

Create a plain node named `keyboard_nudge` (no subclass needed: it only publishes).

```
    pub = node.create_publisher(Int32, '/keyboard/keypress', 10)
```

Publisher for /keyboard/keypress, which the disturbance node listens to.

```
    print(HELP)
```

Show the key help.

```

    settings = termios.tcgetattr(sys.stdin)
```

Save the current terminal settings so they can be restored on exit.

```
    try:
```

Start a block whose `finally` always restores the terminal.

```
        tty.setcbreak(sys.stdin.fileno())
```

Switch to cbreak mode: keys arrive instantly, without Enter.

```
        while rclpy.ok():
```

Loop until ROS shuts down.

```
            key = read_key(0.1)
```

Wait up to 0.1 s for a key.

```
            if key is None:
```

No key pressed...

```
                continue
```

...check again.

```
            if key in ('q', '\x03'):
```

'q' or Ctrl+C (character 3 in cbreak mode)...

```
                break
```

...leaves the loop.

```
            code = key if isinstance(key, int) else ord(key)
```

Arrow keys are already integer codes; other keys are converted to their character code.

```
            if code in NAMES or key in '+=-_':
```

Only arrows and the strength keys are sent.

```
                pub.publish(Int32(data=code))
```

Publish the key code.

```
                if code in NAMES:
```

If it was an arrow...

```
                    print(f'Nudge {NAMES[code]}', flush=True)
```

...print which way the push goes.

```
                else:
```

Otherwise it was a strength key...

```
                    print('Stronger push' if key in '+=' else 'Weaker push', flush=True)
```

...print whether the push got stronger or weaker.

```
76      except KeyboardInterrupt:
```

Ctrl+C can also arrive as an exception...

```
77          pass
```

...which is fine.

```
78      finally:
```

Always run this cleanup, even after an error.

```
79          termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)
```

Restore the original terminal settings; otherwise the terminal would stay in cbreak mode after the program exits.

```
80          node.destroy_node()
```

Release the node.

```
81          rclpy.try_shutdown()
```

Shut ROS down if it is still running.

```
82
83
84  if __name__ == '__main__':
85      main()
```

**Lines 84–85** Run `main` only when the file is executed directly.

# sim.launch.py

`src/omni_pendulum_sim/launch/sim.launch.py` · 113 lines

Starts everything: Gazebo, the ROS-Gazebo bridge, the robot and the nodes.

```
1  import os
```

`os.path.join` builds file paths that work on any system.

```
2
3  from ament_index_python.packages import get_package_share_directory
```

Finds where a ROS package's shared files (launch, urdf, meshes, worlds, config) were installed, e.g. `install/omni_pendulum_sim/share/omni_pendulum_sim`.

```
4  from launch import LaunchDescription
```

`LaunchDescription` is the list of everything this launch file starts.

```
5  from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, TimerAction
```

`DeclareLaunchArgument` creates command-line options such as `gui:=false`; `IncludeLaunchDescription` runs another launch file; `TimerAction` delays an action.

```
6  from launch.conditions import IfCondition, UnlessCondition
```

`IfCondition` / `UnlessCondition` start an action only when a launch argument is true / false.

```
7  from launch.launch_description_sources import PythonLaunchDescriptionSource
```

Tells `IncludeLaunchDescription` that the included file is a Python launch file.

```
8  from launch.substitutions import Command, LaunchConfiguration
```

`Command` runs a shell command at launch time and uses its output; `LaunchConfiguration` reads the value of a launch argument.

```
9  from launch_ros.actions import Node
```

`Node` describes a ROS 2 program to start.

```
10 from launch_ros.parameter_descriptions import ParameterValue
```

`ParameterValue` wraps a value and its type, so the URDF text from xacro is passed as a string parameter (not parsed as YAML).

```
11
12 WORLD_NAME = 'balance_world'
```

Name of the Gazebo world. It must match `<world name=...>` in balance_world.sdf, because the spawn service and the wrench topics contain it.

```
13
14
15 def generate_launch_description():
```

ROS 2 calls this function when you run `ros2 launch omni_pendulum_sim sim.launch.py`.

```
16     pkg_omni_pendulum = get_package_share_directory('omni_pendulum_sim')
```

Install location of this package's shared files.

```
17     pkg_ros_gz_sim = get_package_share_directory('ros_gz_sim')
```

Install location of ros_gz_sim, which provides the standard Gazebo launch file and the `create` spawner.

```
18
19     xacro_file = os.path.join(pkg_omni_pendulum, 'urdf', 'robot.urdf.xacro')
```

Path to the robot description (xacro = XML macros that expand into URDF).

```
20     world = os.path.join(pkg_omni_pendulum, 'worlds', f'{WORLD_NAME}.sdf')
```

Path to the world file (ground, light, physics settings and the ApplyLinkWrench system).

```
21     gui_config = os.path.join(pkg_omni_pendulum, 'config', 'gui.config')
```

Path to the custom Gazebo GUI layout (default layout + Key Publisher + closer camera).

```
22      robot_description = ParameterValue(Command(['xacro ', xacro_file]), value_type=str)
```

Run `xacro robot.urdf.xacro` at launch time; its output, the full URDF text, becomes the `robot_description` parameter. Doing this at launch time means edits to the xacro take effect on the next launch without rebuilding.

```
23
24      gui = LaunchConfiguration('gui')
```

Reads the `gui` launch argument.

```
25      push = LaunchConfiguration('push')
```

Reads the `push` launch argument (disturbance node on/off).

```
26      balance = LaunchConfiguration('balance')
```

Reads the `balance` launch argument (controller on/off).

```
27      declare_gui = DeclareLaunchArgument(
28          'gui', default_value='true', description='Start the Gazebo GUI')
```

**Lines 27–28** Declares `gui` (default true). `gui:=false` runs Gazebo without a window, which was used for all the automated tests.

```
29      declare_push = DeclareLaunchArgument(
30          'push', default_value='true', description='Random nudges, like a person tapping it')
```

**Lines 29–30** Declares `push` (default true). `push:=false` disables nudges entirely.

```
31      declare_balance = DeclareLaunchArgument(
32          'balance', default_value='true',
33          description='Run the balance controller (false = watch the pole fall)')
```

**Lines 31–33** Declares `balance` (default true). `balance:=false` lets you watch the pole fall with no controller, which proves gravity and the free joints work.

```
34
35      node_robot_state_publisher = Node(
```

robot_state_publisher reads the URDF and /joint_states and publishes the TF transform of every link, which tools such as RViz use to display the robot.

```
36          package='robot_state_publisher',
```

The ROS package containing the program.

```
37          executable='robot_state_publisher',
```

The program (executable) name inside that package.

```
38          output='screen',
```

Print its output in the launch terminal.

```
39          parameters=[{'robot_description': robot_description, 'use_sim_time': True}]
```

Give it the URDF text, and tell it to use simulation time from /clock so its transforms carry Gazebo timestamps.

```
40      )
```

End of the Node description.

```
41
42      # Gazebo server and GUI in one process, so the GUI always shows this world;
43      # gui:=false runs the server only (-s)
```

**Lines 42–43** Comment. The server (physics) and GUI (window) used to be started as two separate processes. When an old server was left running in the background, the new GUI attached to the wrong one and the robot was missing from the entity tree. Starting both in one process guarantees the window shows this simulation.

```
44      gz_sim_launch = os.path.join(pkg_ros_gz_sim, 'launch', 'gz_sim.launch.py')
```

Path to ros_gz_sim's standard launch file for Gazebo.

```
    gz_sim = IncludeLaunchDescription(
```

Include that launch file to start Gazebo with the GUI...

```
        PythonLaunchDescriptionSource(gz_sim_launch),
```

...it is a Python launch file...

```
        launch_arguments={'gz_args': f'-r -v 3 {world} --gui-config {gui_config}',
                          'on_exit_shutdown': 'true'}.items(),
```

**Lines 47–48** ...with these Gazebo arguments: `-r` starts the simulation running (not paused), `-v 3` sets log verbosity, then the world file and `--gui-config` with our layout. `on_exit_shutdown` stops the whole launch when the Gazebo window is closed.

```
        condition=IfCondition(gui)
```

...only when `gui` is true.

```
    )
```

End of the include.

```
    gz_server = IncludeLaunchDescription(
```

A second include of the same launch file for headless mode...

```
        PythonLaunchDescriptionSource(gz_sim_launch),
```

...Python launch file...

```
        launch_arguments={'gz_args': f'-r -s -v 3 {world}', 'on_exit_shutdown': 'true'}.items(),
```

...with `-s` (server only, no window).

```
        condition=UnlessCondition(gui)
```

...only when `gui` is false.

```
    )
```

End of the include.

```

    # base_footprint is at ground level; spawn just above it so the landing
    # does not knock the pendulum over
```

**Lines 57–58** Comment: why the spawn height is tiny.

```
    node_spawn_entity = Node(
```

The `create` program from ros_gz_sim asks Gazebo to add the robot to the world.

```
        package='ros_gz_sim',
```

Package containing `create`.

```
        executable='create',
```

Executable name.

```
        output='screen',
```

Show its output.

```
        arguments=['-world', WORLD_NAME, '-topic', 'robot_description',
                   '-name', 'omni_pendulum', '-z', '0.005']
```

**Lines 63–64** Arguments: spawn into `balance_world`, read the robot description from the `robot_description` topic (published by robot_state_publisher), name the model `omni_pendulum` (the disturbance node refers to `omni_pendulum::pendulum_link`), and place it 5 mm above the ground so it settles gently instead of dropping and knocking the pole over.

```
    )
```

End of the spawner Node.

```

    node_ros_gz_bridge = Node(
```

ros_gz_bridge's `parameter_bridge` translates messages between ROS 2 (DDS) and Gazebo Transport, two separate communication systems.

```
68          package='ros_gz_bridge',
```

Package of the bridge.

```
69          executable='parameter_bridge',
```

Executable name.

```
70          arguments=[
```

Each argument is `topic@ROS_type<direction>Gazebo_type`. `]` means ROS to Gazebo, `[` means Gazebo to ROS.

```
71              '/cmd_vel@geometry_msgs/msg/Twist]gz.msgs.Twist',
```

Velocity commands from the controller go to Gazebo's MecanumDrive system.

```
72              '/joint_states@sensor_msgs/msg/JointState[gz.msgs.Model',
```

Joint states from Gazebo come to ROS (Gazebo sends them as a `Model` message; the bridge converts them to `JointState`).

```
73              '/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
```

Wheel-based odometry from MecanumDrive comes to ROS.

```
74              '/ground_truth/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
```

Ground-truth odometry (true pose and velocity from the physics engine) comes to ROS. Added to measure wheel slip.

```
75              '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',
```

The simulation clock comes to ROS. Every node with `use_sim_time: True` uses it instead of the computer's clock.

```
76              '/keyboard/keypress@std_msgs/msg/Int32[gz.msgs.Int32',
```

Key presses from the Gazebo window's Key Publisher plugin come to ROS for the disturbance node.

```
77              f'/world/{WORLD_NAME}/wrench/persistent'
78              '@ros_gz_interfaces/msg/EntityWrench]gz.msgs.EntityWrench',
```

**Lines 77–78** Persistent wrenches (nudges) go from ROS to Gazebo's ApplyLinkWrench system. The f-string and the plain string on the next line are joined by Python into one argument.

```
79              f'/world/{WORLD_NAME}/wrench/clear@ros_gz_interfaces/msg/Entity]gz.msgs.Entity',
```

Wrench-clear commands go from ROS to Gazebo.

```
80          ],
```

End of the argument list.

```
81          parameters=[{'use_sim_time': True}],
```

The bridge stamps messages using simulation time.

```
82          output='screen'
```

Show its output.

```
83      )
```

End of the bridge Node.

```
84
85      node_balance_controller = Node(
```

The balance controller node from this package.

```
86          package='omni_pendulum_sim',
```

This package.

```
87          executable='balance_controller',
```

The `balance_controller` executable (defined in setup.py).

```
88          parameters=[{'use_sim_time': True}],
```

Use simulation time, so the controller's clock matches the physics.

```
89          output='screen',
```

Show its log messages.

```
90          condition=IfCondition(balance)
```

Start it only when `balance` is true.

```
91      )
```

End of the controller Node.

```
92
93      node_disturbance = Node(
```

The disturbance (nudge) node.

```
94          package='omni_pendulum_sim',
```

This package.

```
95          executable='disturbance',
```

The `disturbance` executable.

```
96          parameters=[{'use_sim_time': True, 'world': WORLD_NAME}],
```

Simulation time, and the world name for the wrench topics.

```
97          output='screen',
```

Show its log messages (each nudge is printed).

```
98          condition=IfCondition(push)
```

Start it only when `push` is true.

```
99      )
```

End of the disturbance Node.

```
100
101     return LaunchDescription([
```

Return everything to start, in order.

```
102         declare_gui,
103         declare_push,
104         declare_balance,
```

**Lines 102–104** The three launch arguments must be declared first.

```
105         node_robot_state_publisher,
```

robot_state_publisher (also publishes `robot_description`, which the spawner needs).

```
106         gz_sim,
```

Gazebo with GUI (if gui:=true).

```
107         gz_server,
```

Gazebo headless (if gui:=false). Exactly one of the two starts.

```
108         node_ros_gz_bridge,
```

The bridge.

```
109         node_balance_controller,
```

The controller.

```
110         node_disturbance,
```

The disturbance node.

```
111         # Give Gazebo and robot_state_publisher a moment before spawning
```

Comment.

```
112         TimerAction(period=2.0, actions=[node_spawn_entity]),
```

Spawn the robot after 2 seconds, giving Gazebo time to load the world and robot_state_publisher time to publish the description.

```
113         ])
```

End of the list and function.

# robot.urdf.xacro

`src/omni_pendulum_sim/urdf/robot.urdf.xacro` · 275 lines

The robot model: LIMO chassis, mecanum wheels, platform, pendulum and Gazebo plugins.

```
<?xml version="1.0"?>
```

XML declaration: this file is XML.

```
<!--
  AgileX LIMO in mecanum (omnidirectional) mode carrying a 2-DOF inverted pendulum.

  Chassis/wheel dimensions and meshes come from AgileX's limo_description package
  (https://github.com/agilexrobotics/limo_ros2). That package only ships diff-drive and
  Ackermann Gazebo Classic models, so the mecanum behaviour is added here for Gazebo
  Harmonic with anisotropic wheel friction + the gz MecanumDrive system.
-->
```

**Lines 2–9** Header comment: what the model is and where the LIMO dimensions and meshes came from, and that the mecanum behaviour was added for Gazebo Harmonic because AgileX's package only has diff-drive and Ackermann models for the old Gazebo Classic.

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" xmlns:gz="http://gazebosim.org/schema" name
="omni_pendulum">
```

The root element. `xmlns:xacro` enables xacro macros/properties; `xmlns:gz` declares the Gazebo namespace needed for the `gz:expressed_in` attribute on the wheel friction (without it xacro fails with 'unbound prefix'). `name` is the robot's name.

```

  <xacro:property name="M_PI" value="3.14159265"/>
```

xacro property (a named constant) for pi.

```
  <xacro:property name="mesh_dir" value="file://$(find omni_pendulum_sim)/meshes/limo"/>
```

Folder of the LIMO meshes. `$(find omni_pendulum_sim)` is replaced by the package's install path when xacro runs; `file://` makes it an absolute path both Gazebo and RViz can load.

```

  <!-- LIMO geometry (from limo_four_diff.xacro) -->
```

Comment: the next values come from AgileX's limo_four_diff.xacro.

```
  <xacro:property name="base_x_size" value="0.13"/>
```

Chassis collision box length (m).

```
  <xacro:property name="base_y_size" value="0.12"/>
```

Chassis collision box width (m).

```
  <xacro:property name="base_z_size" value="0.1"/>
```

Original box height, still used for the chassis inertia formula.

```
  <xacro:property name="base_mass" value="2.1557"/>
```

Chassis mass (kg), from AgileX's URDF.

```
  <xacro:property name="base_height" value="0.15"/>
```

Height of base_link above the ground (m).

```
  <xacro:property name="wheelbase" value="0.2"/>
```

Distance between front and rear axles (m).

```
  <xacro:property name="track" value="0.13"/>
```

Distance between the left and right wheel mounting points (m).

```
  <xacro:property name="wheel_vertical_offset" value="-0.10"/>
```
Wheel axle height relative to base_link (m, below it).
```
  <xacro:property name="wheel_length" value="0.045"/>
```
Wheel width (m).
```
  <xacro:property name="wheel_radius" value="0.045"/>
```
Wheel radius (m).
```
  <xacro:property name="wheel_mass" value="0.5"/>
```
Mass of each wheel (kg).
```
  <!-- Height of the LIMO's top surface at the centre of the body, measured from the mesh -->
```
Comment.
```
  <xacro:property name="body_top_height" value="0.1455"/>
```
Height of the LIMO's top surface at the centre, measured from the mesh's vertices (0.1455 m). The body is not flat: the centre is a dip and a sloped cover peaks at 0.2535 m.
```

  <!-- Flat platform on four posts, clearing the LIMO's sloped top cover (peak 0.2535 m) -->
```
Comment: the platform was added so the pole sits on a flat surface above the sloped cover instead of floating.
```
  <xacro:property name="platform_top_height" value="0.266"/>
```
Height of the platform's top surface above the ground (just above the cover's highest point).
```
  <xacro:property name="platform_x_size" value="0.26"/>
```
Platform length (m).
```
  <xacro:property name="platform_y_size" value="0.17"/>
```
Platform width (m).
```
  <xacro:property name="platform_thickness" value="0.008"/>
```
Platform plate thickness (m).
```
  <xacro:property name="platform_mass" value="0.2"/>
```
Platform mass (kg).
```
  <xacro:property name="post_x" value="0.10"/>
```
X position of the support posts (m).
```
  <xacro:property name="post_y" value="0.065"/>
```
Y position of the support posts (m).
```
  <xacro:property name="post_radius" value="0.006"/>
```
Post radius (m).
```
  <xacro:property name="post_bottom_height" value="0.12"/>
```
Height where the posts start (they disappear into the body).
```

  <!-- Pendulum: thin rod with a ball on top -->
```
Comment.
```
  <xacro:property name="mount_thickness" value="0.012"/>
```
Thickness of the small puck under the pivot (m).
```
  <xacro:property name="rod_length" value="0.5"/>
```
Rod length (m).
```
  <xacro:property name="rod_radius" value="0.008"/>
```
Rod radius (m).

```
45   <xacro:property name="rod_mass" value="0.1"/>
```

Rod mass (kg).

```
46   <xacro:property name="bob_radius" value="0.025"/>
```

Radius of the ball on top (m).

```
47   <xacro:property name="bob_mass" value="0.1"/>
```

Mass of the ball (kg).

```
48
49   <link name="base_footprint"/>
```

`base_footprint`: an empty link at ground level under the robot, the conventional root frame for mobile robots.

```
50
51   <joint name="base_joint" type="fixed">
```

A fixed joint rigidly attaches two links.

```
52     <parent link="base_footprint"/>
```

Parent link.

```
53     <child link="base_link"/>
```

Child link.

```
54     <origin xyz="0 0 ${base_height}" rpy="0 0 0"/>
```

The chassis frame sits 0.15 m above the footprint.

```
55   </joint>
```

End of joint.

```
56
57   <link name="base_link">
```

`base_link`: the LIMO chassis.

```
58     <inertial>
```

Mass properties used by the physics engine.

```
59       <mass value="${base_mass}"/>
```

Chassis mass.

```
60       <inertia ixx="${base_mass * (base_y_size**2 + base_z_size**2) / 12}" ixy="0" ixz="0"
61                iyy="${base_mass * (base_x_size**2 + base_z_size**2) / 12}" iyz="0"
62                izz="${base_mass * (base_x_size**2 + base_y_size**2) / 12}"/>
```

**Lines 60–62** Inertia tensor of a solid box: $I = m(a^2 + b^2)/12$ for each axis. The AgileX file had unrealistic values (0.24 to 0.96 kg*m$^2$), so they are computed from the box size instead.

```
63     </inertial>
```

End of inertial.

```
64     <visual>
```

What you see (does not affect physics).

```
65       <origin xyz="0 0 ${-base_height}" rpy="0 0 ${M_PI / 2}"/>
```

The mesh's origin is at ground level and it is modelled along a different axis, so it is moved down 0.15 m and rotated 90 degrees about Z to line up with base_link.

```
66       <geometry><mesh filename="${mesh_dir}/limo_base_lite.dae"/></geometry>
```

The simplified LIMO body mesh (41k triangles instead of 756k, so Gazebo no longer freezes while loading).

```
67     </visual>
```

End of visual.

```
68      <!-- Box sits flush with the body's top surface so it never touches the pendulum -->
```

Comment.

```
69      <collision>
```

The shape used for collisions (a simple box, much cheaper than the mesh).

```
70        <origin xyz="0 0 ${body_top_height - base_height - 0.04}"/>
```

The box is lowered so its top is flush with the body's top surface. AgileX's box stuck 5 cm above the real body and would have hit the pendulum.

```
71        <geometry><box size="${base_x_size} ${base_y_size} 0.08"/></geometry>
```

Box size: original length and width, 8 cm tall.

```
72      </collision>
```

End of collision.

```
73    </link>
```

End of base_link.

```
74
75    <!-- Mecanum wheel. Real mecanum rollers are approximated with anisotropic friction:
76         full grip (mu) along fdir1, zero grip (mu2) perpendicular to it. fdir1 is
77         expressed in base_footprint (the root link) so it does not spin with the wheel. -->
```

**Lines 75–77** Comment describing the mecanum approximation. Real mecanum wheels have angled rollers; simulating dozens of tiny rollers is slow and unstable. Instead each wheel is a plain cylinder whose friction is anisotropic: full grip along one diagonal direction (fdir1) and zero grip across it, which is the net effect of the rollers.

```
78    <xacro:macro name="mecanum_wheel" params="prefix x_reflect y_reflect fdir">
```

A xacro macro is a reusable template; this one creates one complete wheel. Parameters: name prefix, front/rear sign, left/right sign, friction direction.

```
79      <link name="${prefix}_wheel_link">
```

The wheel link.

```
80        <inertial>
```

Mass properties.

```
81          <origin xyz="0 ${wheel_length / 2} 0"/>
```

Centre of mass in the middle of the wheel's width (the joint is at its inner face).

```
82          <mass value="${wheel_mass}"/>
```

Wheel mass.

```
83          <inertia ixx="${wheel_mass * (3 * wheel_radius**2 + wheel_length**2) / 12}" ixy="0" ixz
    ="0"
84                   iyy="${wheel_mass * wheel_radius**2 / 2}" iyz="0"
85                   izz="${wheel_mass * (3 * wheel_radius**2 + wheel_length**2) / 12}"/>
```

**Lines 83–85** Inertia of a solid cylinder whose axis is the link's Y axis: $m*r^2/2$ about the axle, $m(3r^2 + h^2)/12$ about the other two axes.

```
86        </inertial>
```

End of inertial.

```
87        <visual>
```

Visual.

```
88          <geometry><mesh filename="${mesh_dir}/limo_wheel_lite.dae"/></geometry>
```

Simplified LIMO wheel mesh.

```
89        </visual>
```

End of visual.

```
90      <collision>
```
Collision.
```
91        <origin xyz="0 ${wheel_length / 2} 0" rpy="${M_PI / 2} 0 0"/>
```
Shift the cylinder outward by half its width and rotate it 90 degrees so its axis lies along the axle.
```
92        <geometry><cylinder radius="${wheel_radius}" length="${wheel_length}"/></geometry>
```
Cylinder size.
```
93      </collision>
```
End of collision.
```
94    </link>
```
End of wheel link.
```
95
96    <!-- Right wheels are flipped so the mesh faces outward; the axis flips with
97         them, so positive joint velocity always drives the robot forward -->
```
**Lines 96–97** Comment explaining the joint orientation trick below.
```
98    <joint name="${prefix}_wheel_joint" type="continuous">
```
A continuous joint spins without limits (a wheel axle).
```
99      <parent link="base_link"/>
```
Attached to the chassis.
```
100     <child link="${prefix}_wheel_link"/>
```
Child: this wheel.
```
101     <origin xyz="${x_reflect * wheelbase / 2} ${y_reflect * track / 2} ${wheel_vertical_offset}"
```
Wheel position: half the wheelbase forward or back, half the track left or right, below the chassis.
```
102             rpy="${(1 - y_reflect) / 2 * M_PI} 0 0"/>
```
Right wheels (y_reflect = -1) are rotated 180 degrees about X so the mesh faces outward; left wheels are not rotated.
```
103     <axis xyz="0 ${y_reflect} 0"/>
```
The spin axis. For right wheels it is flipped too, so after the 180-degree rotation every wheel's positive spin drives the robot forward, which is what MecanumDrive expects.
```
104   </joint>
```
End of joint.
```
105
106   <gazebo reference="${prefix}_wheel_link">
```
Gazebo-specific settings for this wheel link (URDF itself has no friction model).
```
107     <collision>
```
These settings are merged into the wheel's collision when URDF is converted to SDF.
```
108       <surface>
```
Contact surface properties.
```
109         <friction>
```
Friction.
```
110           <ode>
```
ODE-style friction parameters (also understood by Gazebo Harmonic's DART physics).
```
111             <mu>1.0</mu>
```
`mu` = 1.0: full friction along the primary direction fdir1.
```
112             <mu2>0.0</mu2>
```
`mu2` = 0.0: no friction perpendicular to it, so the wheel slides freely sideways along its rollers.

```
113              <fdir1 gz:expressed_in="base_footprint">${fdir}</fdir1>
```

The primary friction direction, a diagonal such as (1, -1, 0). `gz:expressed_in` makes it fixed in base_footprint; otherwise it would rotate with the spinning wheel. base_footprint is used because base_link is merged into it during URDF-to-SDF conversion (fixed joints are 'lumped').

```
114            </ode>
115          </friction>
116        </surface>
117      </collision>
118    </gazebo>
```

**Lines 114–118** Closing tags.

```
119  </xacro:macro>
```

End of the macro.

```
120
121  <xacro:mecanum_wheel prefix="front_left" x_reflect="1" y_reflect="1" fdir="1 -1 0"/>
122  <xacro:mecanum_wheel prefix="front_right" x_reflect="1" y_reflect="-1" fdir="1 1 0"/>
123  <xacro:mecanum_wheel prefix="rear_left" x_reflect="-1" y_reflect="1" fdir="1 1 0"/>
124  <xacro:mecanum_wheel prefix="rear_right" x_reflect="-1" y_reflect="-1" fdir="1 -1 0"/>
```

**Lines 121–124** Create the four wheels. The friction diagonals form an X pattern when seen from above, like real mecanum rollers; this is what lets the robot strafe sideways. These directions were verified in simulation by driving forward, sideways and diagonally.

```
125
126  <!-- Platform: origin at the centre of the plate's top surface -->
```

Comment.

```
127  <xacro:property name="post_length" value="${platform_top_height - platform_thickness - post_bo
ttom_height}"/>
```

Post length: from their starting height up to the underside of the plate.

```
128  <xacro:macro name="platform_post" params="x y">
```

Macro producing one post visual at (x, y).

```
129    <visual>
```

Visual only (the posts have no collision).

```
130      <origin xyz="${x} ${y} ${-platform_thickness - post_length / 2}"/>
```

Centre the post halfway between its bottom and the underside of the plate.

```
131      <geometry><cylinder radius="${post_radius}" length="${post_length}"/></geometry>
```

Post shape.

```
132      <material name="post_mat"><color rgba="0.7 0.7 0.72 1.0"/></material>
```

Grey colour.

```
133    </visual>
```

End of visual.

```
134  </xacro:macro>
```

End of macro.

```
135
136  <link name="platform_link">
```

The platform link.

```
137    <inertial>
```

Mass properties.

```
138      <origin xyz="0 0 ${-platform_thickness / 2}"/>
```

Centre of mass in the middle of the plate.

```
139       <mass value="${platform_mass}"/>
```

Platform mass.

```
140       <inertia ixx="${platform_mass * platform_y_size**2 / 12}" ixy="0" ixz="0"
141                iyy="${platform_mass * platform_x_size**2 / 12}" iyz="0"
142                izz="${platform_mass * (platform_x_size**2 + platform_y_size**2) / 12}"/>
```

**Lines 140–142** Inertia of a thin plate.

```
143     </inertial>
```

End of inertial.

```
144     <visual>
```

Plate visual.

```
145       <origin xyz="0 0 ${-platform_thickness / 2}"/>
```

Centred half a thickness below the link origin (the origin is the plate's top surface).

```
146       <geometry><box size="${platform_x_size} ${platform_y_size} ${platform_thickness}"/></geometry>
```

Box shape of the plate.

```
147       <material name="platform_mat"><color rgba="0.95 0.75 0.1 1.0"/></material>
```

Yellow colour.

```
148     </visual>
```

End of visual.

```
149     <collision>
```

The plate has collision, so it physically supports the pendulum mount.

```
150       <origin xyz="0 0 ${-platform_thickness / 2}"/>
```

Same position.

```
151       <geometry><box size="${platform_x_size} ${platform_y_size} ${platform_thickness}"/></geometry>
```

Same box.

```
152     </collision>
```

End of collision.

```
153     <xacro:platform_post x="${post_x}" y="${post_y}"/>
154     <xacro:platform_post x="${post_x}" y="${-post_y}"/>
155     <xacro:platform_post x="${-post_x}" y="${post_y}"/>
156     <xacro:platform_post x="${-post_x}" y="${-post_y}"/>
```

**Lines 153–156** Four posts at the corners.

```
157   </link>
```

End of platform link.

```
158
```

```
159   <joint name="platform_joint" type="fixed">
```

Fixed joint attaching the platform to the chassis.

```
160     <parent link="base_link"/>
```

Parent: chassis.

```
161     <child link="platform_link"/>
```

Child: platform.

```
162     <origin xyz="0 0 ${platform_top_height - base_height}"/>
```

Platform top at 0.266 m above the ground (base_link is at 0.15 m).

```
163   </joint>
```

End of joint.

```
164
165   <!-- Pendulum mount: a small puck on the platform; the pivot is on top of it -->
```
Comment.
```
166   <link name="pendulum_mount">
```
The puck under the pivot.
```
167     <inertial>
```
Mass properties (small).
```
168       <mass value="0.05"/>
```
50 g.
```
169       <inertia ixx="0.00001" ixy="0" ixz="0" iyy="0.00001" iyz="0" izz="0.00001"/>
```
Tiny inertia.
```
170     </inertial>
```
End of inertial.
```
171     <visual>
```
Visual.
```
172       <origin xyz="0 0 ${-mount_thickness / 2}"/>
```
Centred half its thickness below the pivot.
```
173       <geometry><cylinder radius="0.02" length="${mount_thickness}"/></geometry>
```
Small cylinder.
```
174       <material name="mount_mat"><color rgba="0.2 0.2 0.2 1.0"/></material>
```
Dark grey.
```
175     </visual>
```
End of visual.
```
176   </link>
```
End of link.
```
177
178   <joint name="pendulum_mount_joint" type="fixed">
```
Fixed joint putting the puck on the platform.
```
179     <parent link="platform_link"/>
```
Parent: platform.
```
180     <child link="pendulum_mount"/>
```
Child: puck.
```
181     <origin xyz="0 0 ${mount_thickness}"/>
```
The puck's origin (the pivot point) is one puck-thickness above the plate.
```
182   </joint>
```
End of joint.
```
183
184   <!-- Pitch Joint: positive angle tips the pendulum toward +X -->
```
Comment.
```
185   <joint name="pendulum_pitch_joint" type="revolute">
```
Pitch joint: a revolute (hinge) joint, the first of two hinges that let the pole tip in any direction.
```
186     <parent link="pendulum_mount"/>
```
Parent: the puck.
```
187     <child link="dummy_pitch_link"/>
```
Child: an intermediate link.

```
188     <origin xyz="0 0 0" rpy="0 0 0"/>
```
At the pivot point.
```
189     <axis xyz="0 1 0"/>
```
Rotates about Y, so the pole tips forward/back (positive = toward +X by the right-hand rule).
```
190     <limit lower="-1.57" upper="1.57" effort="10.0" velocity="10.0"/>
```
Limits of plus/minus 90 degrees, so a fallen pole rests on its side instead of passing through the robot. Effort and velocity limits are required by URDF.
```
191     <dynamics damping="0.0001" friction="0.0"/>
```
Almost no damping and no friction: the hinge is essentially free, so only gravity, nudges and the robot's motion move the pole.
```
192   </joint>
```
End of joint.
```
193
194   <link name="dummy_pitch_link">
```
Intermediate link between the two hinges. Two perpendicular hinges in series make a universal joint (2 degrees of freedom).
```
195     <inertial>
```
Every link that moves needs mass properties...
```
196       <mass value="0.001"/>
```
...1 g...
```
197       <inertia ixx="0.00001" ixy="0" ixz="0" iyy="0.00001" iyz="0" izz="0.00001"/>
```
...and tiny inertia, so it does not affect the dynamics.
```
198     </inertial>
```
End of inertial.
```
199   </link>
```
End of link.
```
200
201   <!-- Roll Joint: positive angle tips the pendulum toward -Y -->
```
Comment.
```
202   <joint name="pendulum_roll_joint" type="revolute">
```
Roll joint: the second hinge.
```
203     <parent link="dummy_pitch_link"/>
```
Parent: the intermediate link.
```
204     <child link="pendulum_link"/>
```
Child: the pendulum.
```
205     <origin xyz="0 0 0" rpy="0 0 0"/>
```
Same pivot point.
```
206     <axis xyz="1 0 0"/>
```
Rotates about X, so the pole tips left/right (positive = toward -Y by the right-hand rule).
```
207     <limit lower="-1.57" upper="1.57" effort="10.0" velocity="10.0"/>
```
Same limits as pitch.
```
208     <dynamics damping="0.0001" friction="0.0"/>
```
Same near-zero damping.
```
209   </joint>
```
End of joint.

```
210
211   <!-- Rod + ball combined into one rigid link; inertia is about the combined COM -->
```

Comment.

```
212   <xacro:property name="pend_mass" value="${rod_mass + bob_mass}"/>
```

Total pendulum mass: 0.2 kg.

```
213   <xacro:property name="pend_com" value="${(rod_mass * rod_length / 2 + bob_mass * rod_length) /
pend_mass}"/>
```

Combined centre of mass height: (0.1*0.25 + 0.1*0.5)/0.2 = 0.375 m above the pivot.

```
214   <xacro:property name="pend_ixx" value="${rod_mass * rod_length**2 / 3 + bob_mass * rod_length*
*2
215                                          + 0.4 * bob_mass * bob_radius**2 - pend_mass * pend_co
m**2}"/>
```

**Lines 214–215** Inertia about the combined centre of mass: rod about its end ($mL^2/3$) plus ball about the pivot ($mL^2 + 2mr^2/5$), minus total mass times COM height squared (parallel-axis theorem in reverse). Getting this right matters because the controller gains were computed from the same numbers (J = 0.0334 kg*$m^2$ about the pivot).

```
216   <link name="pendulum_link">
```

The pendulum link (rod + ball as one rigid body).

```
217     <inertial>
```

Mass properties.

```
218       <origin xyz="0 0 ${pend_com}"/>
```

Centre of mass position.

```
219       <mass value="${pend_mass}"/>
```

Total mass.

```
220       <inertia ixx="${pend_ixx}" ixy="0" ixz="0" iyy="${pend_ixx}" iyz="0"
221                izz="${rod_mass * rod_radius**2 / 2 + 0.4 * bob_mass * bob_radius**2}"/>
```

**Lines 220–221** Inertia tensor: equal about X and Y (symmetric pole), very small about the pole's own axis (Z).

```
222     </inertial>
```

End of inertial.

```
223     <visual>
```

Rod visual.

```
224       <origin xyz="0 0 ${rod_length / 2}"/>
```

Centred halfway up the rod.

```
225       <geometry><cylinder radius="${rod_radius}" length="${rod_length}"/></geometry>
```

Cylinder.

```
226       <material name="rod_mat"><color rgba="0.9 0.1 0.1 1.0"/></material>
```

Red.

```
227     </visual>
```

End.

```
228     <visual>
```

Ball visual.

```
229       <origin xyz="0 0 ${rod_length}"/>
```

At the top of the rod.

```
230       <geometry><sphere radius="${bob_radius}"/></geometry>
```

Sphere.

```
231       <material name="bob_mat"><color rgba="0.1 0.3 0.9 1.0"/></material>
```

Blue.

```
232     </visual>
```

End.

```
233     <collision>
234       <origin xyz="0 0 ${rod_length / 2}"/>
235       <geometry><cylinder radius="${rod_radius}" length="${rod_length}"/></geometry>
236     </collision>
237     <collision>
238       <origin xyz="0 0 ${rod_length}"/>
239       <geometry><sphere radius="${bob_radius}"/></geometry>
240     </collision>
```

**Lines 233–240** Collision shapes matching the visuals, so the pole can hit the ground or the robot when it falls.

```
241   </link>
```

End of pendulum link.

```
242
243   <!-- Gazebo Plugins -->
```

Comment.

```
244   <gazebo>
```

A `<gazebo>` block with no reference applies to the whole model: it holds Gazebo system plugins.

```
245     <plugin filename="gz-sim-mecanum-drive-system" name="gz::sim::systems::MecanumDrive">
```

MecanumDrive: turns a body velocity command (/cmd_vel) into the four wheel speeds using mecanum inverse kinematics, and publishes wheel odometry.

```
246       <front_left_joint>front_left_wheel_joint</front_left_joint>
247       <front_right_joint>front_right_wheel_joint</front_right_joint>
248       <back_left_joint>rear_left_wheel_joint</back_left_joint>
249       <back_right_joint>rear_right_wheel_joint</back_right_joint>
```

**Lines 246–249** Which joint is which wheel.

```
250       <wheel_separation>${track + wheel_length}</wheel_separation>
```

Distance between left and right wheel centres: track + one wheel width = 0.175 m.

```
251       <wheel_base>${wheelbase}</wheel_base>
```

Distance between front and rear wheels.

```
252       <wheel_radius>${wheel_radius}</wheel_radius>
```

Wheel radius, needed to convert m/s to rad/s.

```
253       <topic>/cmd_vel</topic>
```

Listen for velocity commands on /cmd_vel. Absolute topic names (leading /) avoid Gazebo prefixing them with the model name.

```
254       <odom_topic>/odom</odom_topic>
```

Publish wheel odometry on /odom.

```
255       <tf_topic>/tf</tf_topic>
```

Publish the odom-to-base transform on /tf (inside Gazebo; not bridged).

```
256       <frame_id>odom</frame_id>
```

Odometry frame name.

```
257       <child_frame_id>base_footprint</child_frame_id>
```

Robot frame name.

```
258     </plugin>
```

End of plugin.

```
259
260     <plugin filename="gz-sim-joint-state-publisher-system" name="gz::sim::systems::JointStatePublisher">
```

JointStatePublisher: publishes every joint's position and velocity.

```
261       <topic>/joint_states</topic>
```

On /joint_states. This is the controller's only sensor input.

```
262     </plugin>
```

End of plugin.

```
263
264     <!-- True pose/velocity of the robot (the mecanum odometry above is computed
265          from wheel speeds and cannot see wheel slip) -->
```

**Lines 264–265** Comment explaining why ground-truth odometry was added.

```
266     <plugin filename="gz-sim-odometry-publisher-system" name="gz::sim::systems::OdometryPublisher">
```

OdometryPublisher: publishes the robot's true pose and velocity straight from the physics engine.

```
267       <odom_topic>/ground_truth/odom</odom_topic>
```

Topic name.

```
268       <odom_frame>world</odom_frame>
```

Fixed world frame.

```
269       <robot_base_frame>base_footprint</robot_base_frame>
```

Robot frame.

```
270       <odom_publish_frequency>100</odom_publish_frequency>
```

100 Hz.

```
271       <dimensions>2</dimensions>
```

Planar robot: only x, y and yaw.

```
272       <tf_topic>/ground_truth/tf</tf_topic>
```

Its transform goes on a separate topic so it does not clash with /tf.

```
273     </plugin>
```

End of plugin.

```
274   </gazebo>
```

End of the gazebo block.

```
275 </robot>
```

End of the robot.

## setup.py

`src/omni_pendulum_sim/setup.py` · 35 lines

Tells colcon how to build and install the package and its programs.

```
from glob import glob
```

`glob` finds files matching a pattern, e.g. every .launch.py file.

```
import os
```

`os.path.join` for building install paths.

```

```

```
from setuptools import find_packages, setup
```

setuptools is how Python packages are built and installed; ROS 2 Python packages (`ament_python`) use it.

```

```

```
package_name = 'omni_pendulum_sim'
```

The package name, used several times below.

```

```

```
setup(
```

`setup(...)` describes how to build and install the package. `colcon build` runs it.

```
    name=package_name,
```

Package name.

```
    version='0.0.0',
```

Version.

```
    packages=find_packages(exclude=['test']),
```

Install the Python module folder `omni_pendulum_sim/` (the three nodes), skipping tests.

```
    data_files=[
```

`data_files`: non-Python files to copy into the install folder. Each entry is (destination folder, list of files).

```
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
```

A marker file that registers the package with the ROS 2 index, so `ros2 run` and `ros2 launch` can find it. Without it you get 'Package not found'.

```
        ('share/' + package_name, ['package.xml']),
```

Install package.xml (package metadata and dependencies).

```
        (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
```

Install all launch files into share/omni_pendulum_sim/launch.

```
        (os.path.join('share', package_name, 'urdf'), glob('urdf/*')),
```

Install the URDF/xacro.

```
        (os.path.join('share', package_name, 'meshes', 'limo'), glob('meshes/limo/*_lite.dae')),
```

Install only the simplified LIMO meshes (`*_lite.dae`, about 4.7 MB). The original 64 MB meshes stay in the source folder for reference.

```
        (os.path.join('share', package_name, 'worlds'), glob('worlds/*.sdf')),
```

Install the world file.

```
        (os.path.join('share', package_name, 'config'), glob('config/*')),
```

Install the Gazebo GUI layout.

```
    ],
```

End of data_files.

```
21      install_requires=['setuptools'],
```

Python packages needed to install this one.

```
22      zip_safe=True,
```

The package can be installed as a compressed archive.

```
23      maintainer='user',
```

Maintainer name (placeholder).

```
24      maintainer_email='user@todo.todo',
```

Maintainer email (placeholder).

```
25      description='2D Inverted Pendulum Omnidirectional Robot Simulation',
```

Short description.

```
26      license='Apache-2.0',
```

License.

```
27      tests_require=['pytest'],
```

Packages needed to run tests.

```
28      entry_points={
```

Entry points: command names created by the install.

```
29          'console_scripts': [
```

`console_scripts` : each line creates an executable that `ros2 run` can start.

```
30              'balance_controller = omni_pendulum_sim.balance_controller:main',
```

`ros2 run omni_pendulum_sim balance_controller` calls `main()` in balance_controller.py.

```
31              'disturbance = omni_pendulum_sim.disturbance:main',
```

`disturbance` calls `main()` in disturbance.py.

```
32              'keyboard_nudge = omni_pendulum_sim.keyboard_nudge:main',
```

`keyboard_nudge` calls `main()` in keyboard_nudge.py.

```
33          ],
34      },
35  )
```

**Lines 33–35** Closing brackets.